

The document becomes code, and the code runs

The win: you can explain, to an interviewer or to yourself, how a tree of nodes becomes one file whose bytes never surprise anyone, how one container per document is started exactly once even when ten callers ask at the same moment, why a child process needs an end from outside and what Node 24 does when you ask for one, how a page framed under an opaque origin is served and healed, and why a reaper must look before it asks; and you can point at the line of code for every claim.

How to use this page. Every code block is cut from the repo at commit ab92728, so what you read is what runs. It is long: one section a day is a fine pace. Sections 1, 3, 4, 6 and 8 are the ones an architect interview will dig into.

```
./init.sh                                # Postgres + Redis + api + sync +
worker + worker-sandbox; builds noon-sandbox:dev
docker compose logs -f worker-sandbox    # "worker draining queues: sandbox",
then each preview job
pnpm exec vitest run apps/worker/src/sandbox.int.test.ts  # real Docker: starts, races,
deadlines, the reaper
pnpm exec playwright test e2e/preview.spec.ts            # the canvas, the real sandbox
worker, a container killed under it
```

Try it: open a document, press *Show the running page*, add a Button and type a label; it appears in the frame within a moment and the frame is not reloaded. Then `docker rm --force noon-sandbox-<document id>` and watch "Rebuilding the preview..." turn back into your page, on its own. Never run two Docker-using suites at once: each one's cleanup removes the other's containers.

The map: a document becomes code, and code becomes a running page

- 1 · One document, one file, the same bytes
- 2 · A container per document, ready in a second
- 3 · Uniqueness is not ownership
- 4 · A child process needs an end from outside
- 5 · The sandbox queue: a job is the right to a container
- 6 · One queue per process, one pool per stack, a reaper with an owner
- 7 · The canvas frames the page
- 8 · The opaque origin
- 9 · Testing a program that runs containers
- 10 · What the type system could not see, epic 4 edition

Your turn: three drills

Recall, then test yourself

The map: a document becomes code, and code becomes a running page

Epic 4 closes half of keystone 8: **one document is one generated TSX file of a fixed shape, and the same document is always the same bytes**. The other half, reading a file back into ops, is epic 5. What this epic adds around that one pure function is a running page: a container per document that serves the file with the customer's own dev server, a job that keeps the file in step with the document, and a frame on the canvas that shows it. Nothing in the room changed. The preview is one more peer, and it never writes.

STEP	WHERE	WHAT HAPPENS
1	packages/codegen/src/index.ts	<code>generate(doc, manifest)</code> : total, deterministic, one file whose only export is <code>Page</code>
2	apps/api/src/app.ts	<code>POST /documents/:id/preview</code> : one <code>INSERT</code> under a per-org cap; the job row IS the preview; 201 even when Redis is away
3	apps/worker/src/main.ts	<code>WORKER_QUEUE=sandbox</code> : the only process holding the Docker socket drains this queue, four... eight at a time
4	apps/worker/src/preview.ts	the handler joins the room as a silent peer, projects from <code>peer.confirmed</code> , and loops: alive? moved? anyone here?
5	apps/worker/src/sandbox.ts	<code>startSandbox</code> : one container per document, single-flight, port chosen once, ready probed from inside
6	apps/worker/sandbox/Dockerfile	baked <code>node_modules</code> , the container's own clone on <code>noon/<id></code> , a preview entry of ours wrapped around the customer's Vite
7	apps/worker/src/sandbox.ts	<code>pushPage</code> : <code>docker exec cat > "\$PAGE"</code> , ~20 ms; Vite hot-updates, React keeps its state
8	apps/web/src/Preview.tsx	asks <code>GET .../preview</code> once a second, never caches the URL; an iframe with an opaque origin, keyed on the URL
9	apps/worker/src/sandbox.ts	<code>reapSandboxes</code> : every 30 s, its own pool only, list Docker first and ask Postgres second

Four ideas carry the epic. **Choose the literal from the value, never from what a schema claims** (section 1). **Uniqueness is not ownership** (section 3). **A child process, like a promise, needs an end from outside** (section 4). And **every resource needs an owner label before it gets a reaper** (section 6). Sections 7 and 8 are about a browser security boundary that broke the design twice before it held.

THIS REPO	JAVA WORLD	PYTHON WORLD
a pure function from a tree to one source file	JavaPoet, or a template engine with a sorted walk	<code>ast.unparse</code> , or Jinja2 with the same discipline
a dev server in a container per document, driven over the docker CLI	Testcontainers' <code>GenericContainer</code> ; <code>docker-java</code>	<code>testcontainers-python</code> ; the <code>docker</code> SDK
a file pushed with <code>docker exec</code> on stdin	<code>ProcessBuilder</code> and its <code>OutputStream</code>	<code>subprocess.run(input=...)</code>
a reaper on a timer, scoped by a label	a <code>@Scheduled</code> cleanup; Testcontainers' ryuk with a session label	a beat task; the same label idea

1 · One document, one file, the same bytes

The first decision is what *deterministic* has to mean here. Not "the same on my machine today": epic 5 opens a pull request whose file must equal a fresh generation, and a sandbox pushes the file into a container on every confirmed op. A generator that sorted differently on two machines would show a diff that nobody made.

FROM `packages/codegen/src/index.ts`

```
/**
 * Keystone 8: one document becomes ONE generated TSX file of a fixed shape, and the same document
 * always generates the same bytes. Determinism is not tidiness here: epic 5 opens a pull request
 * whose file must equal a fresh generation, and epic 4's sandbox pushes this file into a running
 * container. A generator that sorted differently on two machines would show a diff nobody made.
 *
 * So: the tree is walked from `rootId` (never the key order of `doc.nodes`), props are written in
 * name order, and nothing here reads the clock, the random number generator or the environment.
 *
 * Generate from `peer.confirmed`, never from the optimistic document: the optimistic tree holds ops
 * the server has not accepted and may still refuse.
 */
```

So the walk starts at `rootId` and never at the key order of `doc.nodes`; props are written in name order; the import line is sorted; and nothing reads the clock, the random generator or the environment. The walk is an explicit stack rather than recursion, for a reason that is not style: a document restored from a snapshot can be deeper than any call stack, and the room's depth cap is the room's rule, not this function's.

FROM `packages/codegen/src/index.ts`

```
// An explicit stack, not recursion: `maxDepth` is the room's rule, not this function's, and a
// document restored from a snapshot can be deeper than any call stack.
const stack: Frame[] = [{ open: doc.rootId, depth: 0 }];

while (stack.length > 0) {
  const frame = stack.pop();
  if (!frame) break;
  const indent = INDENT.repeat(Math.min(frame.depth + 2, MAX_INDENT_DEPTH));
  if ("close" in frame) {
    lines.push(`${indent}</${frame.close}>`);
  }
```

```

    continue;
  }

```

The file exports the page component and nothing else. That is not tidiness either. The sandbox learning test measured that Vite's Fast Refresh keeps React state across an edit only while a module exports components alone; one extra export (a `BUILD_ID`, say) turns every edit into a full page reload, and E4.3's promise of "within 3 s without a reload" rests on this line:

FROM `packages/codegen/src/index.ts`

```

const importLine = used.size > 0 ? [`import { ${[...used].sort().join(", ")} } from
"${DESIGN_SYSTEM}";`, ""] : [];
// The ONLY export is the page component. Fast Refresh keeps React state across an edit only while
// a module exports components alone; one extra export turns every edit into a full page reload,
// which is the mechanism E4.3's "an edit shows within 3 s without a full reload" rests on.
return { ok: true, tsx: [BANNER, "", ...importLine, "export function Page() {", "  return (",
...lines, " );", "}", ""].join("\n") };
}

```

FROM `packages/codegen/src/index.test.ts`

```

test("the file exports the page component and nothing else", () => {
  const doc = build(add("s", ROOT_ID), add("b", "s", "Button", { label: "Pay" }));
  // Fast Refresh keeps React state only while the module exports components alone: one extra
  // export (a BUILD_ID, say) turns every edit into a full page reload (learning-tests/sandbox).
  const exports = tsx(doc).match(/^export\b.*$/gm) ?? [];
  expect(exports).toEqual(["export function Page() {"]);
});

```

Refusing is the point. A document can outlive its design system: a component dropped, a prop renamed, a type changed. Generating *without* the prop would compile, and epic 5 would read the file back as a document with that prop deleted. So every drift has a name, and the function is total: it never throws, because its caller is a queue handler, and an exception there is a job that fails as `internal` while a reason is something a person can be shown.

FROM `packages/codegen/src/index.ts`

```

// ponytail: not exported until a caller needs to switch on it; Generated carries it either way.
type CodegenReason = PropProblem | "malformed_doc" | "unknown_component" | "reserved_component" |
"parent_takes_no_children";
/**
 * Either the file or the reason there is none. Refusing is the point: a document whose design
 * system has changed under it (a prop that no longer exists, a component that was dropped) must not
 * be quietly generated WITHOUT that prop. Epic 5 reads the file back into a tree, so a silent drop
 * here is a deletion from the document on the round trip.
 *
 * It never throws. The caller is a queue handler: an exception there is a job that fails as
 * `internal`, while a reason is something the user can be shown.
 */
export type Generated = { ok: true; tsx: string } | { ok: false; reason: CodegenReason; detail:
string };

```

FROM packages/codegen/src/index.ts

```
export function generate(doc: Doc, manifest: Manifest): Generated {
  const no: Refuse = (reason, detail) => ({ ok: false, reason, detail });
  // The safety net. checkDoc below names every malformed shape it knows; this is what makes
  // "it returns a reason" true of ANY input rather than only of the inputs somebody listed –
  // a `nodes` that is null, a `props` that is null, a props bag whose getter throws. Writing
  // those three as named guards instead was two more lines that no test could tell apart from
  // this one, so they are gone: the net is the promise, and checkDoc is the diagnosis.
  try {
    return project(doc, manifest, no);
  } catch (err) {
    // `detail` is for the log; the user only ever sees the reason.
    return no("malformed_doc", describe(err));
  }
}

/** The net must not tear on what it catches: `String(x)` and even `instanceof` can throw. */
function describe(err: unknown): string {
  try {
    const message: unknown = err instanceof Error ? err.message : String(err);
    return typeof message === "string" ? message : "an exception";
  } catch {
    return "an exception that cannot be described";
  }
}
```

The property test is the one that kills every future silent drop at once: over four thousand seeded ops, whenever a document generates, the file names exactly the props each node holds.

FROM packages/codegen/src/index.test.ts

```
test("whenever a document is generated, the file names exactly the props the document holds", () => {
  const random = seeded(20260921);
  let doc = emptyDoc();
  let generated = 0;
  for (let i = 0; i < 4000; i++) {
    const op = randomOp(random, doc);
    if (validate(doc, op, manifest).ok) doc = applyOp(doc, op);
    if (i % 20 !== 0) continue;
    const result = generate(doc, manifest);
    if (!result.ok) throw new Error(`${result.reason}: ${result.detail}`); // a valid document must
    always generate
    generated++;
    for (const node of Object.values(doc.nodes)) {
      if (node.parentId === null) continue;
      const line = result.tsx.split("\n").find((l) => l.includes(`data-node-id="${node.id}"`));
      const written = [...(line ?? "").matchAll(/ ([A-Za-z_$][A-Za-z0-9_$]*)=\{\/gu\}).map((m) =>
m[1])];
      expect(written.sort()).toEqual(Object.keys(node.props).sort());
    }
  }
  expect(generated).toBeGreaterThan(100);
});
```

And F13 is proven the way a customer's CI would prove it: the generated file is written beside the sample app's own pages and the sample app's own `tsc` is run on it.

FROM packages/codegen/src/sample-app.int.test.ts

```
// F13: what the canvas generates must compile against the customer's own design system. The sample
// app is that customer repo, outside this workspace, so it is proven the way its own CI would.
const run = promisify(execFile);
const APP = new URL("../seed/sample-app/", import.meta.url).pathname;
const pnpm = (...args: string[]) => run("pnpm", ["--ignore-workspace", ...args], { cwd: APP, timeout:
170_000 });
// Generated pages live beside Showcase.tsx so the import path in the file is the real one. The name
// ends in .tmp.tsx, which seed/sample-app/.gitignore holds, so a crashed run cannot leave a commit
behind.
const GENERATED = `${APP}src/pages/Generated.tmp.tsx`;
```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

`JSON.stringify(value)` as a JavaScript
string literal

JavaPoet's `$S` ;
`StringEscapeUtils`

`json.dumps` ; `repr` for
Python source

`String(-0)` is `"0"` : a different number

`Double.toString(-0.0)` is
`"-0.0"` : the sign survives

`repr(-0.0)` is `'-0.0'` : the
sign survives

`typeof x === "string"` before anything
becomes code

the static type, or `instanceof`
`String`

`isinstance(x, str)`

`Object.keys` misses non-enumerable and
Symbol keys; `Reflect.ownKeys` sees
them

reflection sees private fields that
getters do not show

`vars()` misses `__slots__`
and properties

`push(...arr)` overflows the stack at
about 130 000 arguments

`varargs` build one array: heap-
bound, not stack-bound

`f(*args)` : no fixed cap, but
the C stack still has one

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A lying `toString` answered "Button" to the identifier check and code to the template; the file ran `alert(1)`.

The E4.1 verifier built a manifest whose component name was an object with a `toString` that answers "Button" the first time and `Button />; alert(1); <Button` the second. The identifier regex saw the first answer; the template literal saw the second. The lesson is general: *a value converted to a string twice can answer twice*. Anything that becomes code is checked with `typeof` before it is looked at, and every literal is chosen from the value's own type, never from what the manifest says the value is. Asking `checkProp` and believing it would turn a prop type nobody wrote a case for into a way to write arbitrary source.

FROM `packages/codegen/src/index.ts`

```
// A string, or nothing: a value that is converted to a string once for the check and
again for
// the file can answer differently each time (a lying toString put `alert(1)` in the
file).
if (typeof node.component !== "string") return no("malformed_doc", `${id}: the component
name is not a string`);
```

FROM `packages/codegen/src/index.ts`

```
// The literal is chosen from the VALUE, never from what the manifest says the value is.
// Asking checkProp and believing it turns a prop type nobody wrote a case for into a
way to
// write arbitrary source: `String(anObjectWithAToString)` would land in the file
verbatim.
const literal = jsLiteral(value);
if (literal === undefined) return no("malformed_doc", `${id}: ${node.component}.${key}
is not a value this generator can write`);
attrs.push(`${key}=${literal}`);
}
```

FROM `packages/codegen/src/index.test.ts`

```
test("a component name that is not a string cannot say one thing to the check and another to the
file", () => {
  // Converted to a string once for the identifier check and again for the file: a toString that
  // answers "Button" first and code after that got through, and the file ran `alert(1)`.
  let calls = 0;
  const liar = { toString: () => (calls++ < 1 ? "Button" : "Button />; alert(1); <Button") } as
unknown as string;
  const hostile: Manifest = { version: 1, components: [{ name: liar, acceptsChildren: false,
props: [{ name: "label", type: { kind: "string" }, required: true }] }];
  expect(refusal(childOf("b", liar, { label: "x" }), hostile)).toBe("malformed_doc");
});
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

-0 was written as **0**, and **NaN** would have compiled.

`String(-0)` is `"0"`: a different number, and `validate()` accepts `-0`, so the file must carry it.

`String(NaN)` is the bare identifier `NaN`, which compiles and is not the document's value; so is `Infinity`. Every value is a JSX *expression*, never a quoted attribute, because JSX string attributes have no backslash escapes. The whole of the value-to-source rule fits in one function, and its final fallback is documented as unreachable rather than faked as tested:

FROM `packages/codegen/src/index.ts`

```
/**
 * One prop value as JavaScript source, or `undefined` when there is no honest way to write it.
 *
 * Every value is a JSX EXPRESSION, never a quoted attribute: JSX string attributes have no
 * backslash escapes, so `label="a \n b"` would put those two characters in the page. A JSON
 * literal
 * is legal JavaScript for every string the contract allows. A number that is not finite is
 * refused
 * rather than written, because `String(NaN)` is the bare identifier `NaN`: it compiles, and it
 * is
 * not the document's value.
 *
 * The final `return undefined` is belt and braces behind checkProp, which already refuses every
 * value that is not a string, number or boolean: a mutation test cannot reach it. It stays
 * because
 * the alternative fallback, String(value), is how an object's toString becomes source code.
 */
function jsLiteral(value: unknown): string | undefined {
  if (typeof value === "string") return JSON.stringify(value);
  // String(-0) is "0": a different number. validate() accepts -0, so it is written, not
  refused.
  if (typeof value === "number") return Number.isFinite(value) ? (Object.is(value, -0) ? "-0" :
String(value)) : undefined;
  if (typeof value === "boolean") return String(value);
  return undefined;
}
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A hidden prop was dropped while the file was called complete.

`Object.keys` lists the enumerable string keys. `Object.hasOwn`, which the required-prop check uses, sees a non-enumerable one too. So a prop defined with `enumerable: false`, or keyed by a `Symbol`, passed the required check and vanished from the file: a silent drop, the one thing this module exists to prevent. One comparison refuses both: `Reflect.ownKeys` counts every own key, Symbols included.

FROM `packages/codegen/src/index.ts`

```
// Every OWN key, not only the enumerable strings: `Object.hasOwn` (the required-prop
check
// below) sees a hidden prop, so a walk that did not would drop it while calling the file
// complete. `Reflect.ownKeys` also counts Symbol keys, so one comparison refuses both.
if (Reflect.ownKeys(node.props).length !== Object.keys(node.props).length) return
no("malformed_doc", `${id}: a prop is hidden (not enumerable, or keyed by a Symbol)`);
```

FROM `packages/codegen/src/index.test.ts`

```
test("a prop that is not enumerable is refused, not dropped: the file would lose a prop the
document holds", () => {
  const props = Object.defineProperty({}, "label", { value: "hi", enumerable: false }) as
Record<string, PropValue>;
  expect(refusal(childOf("b", "Button", props))).toBe("malformed_doc");
});

test("a prop keyed by a Symbol is refused, not dropped", () => {
  expect(refusal(withProps("s", { [Symbol("gap")]: 4 }))).toBe("malformed_doc");
});
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A well-formed document with 200 000 children was called malformed.

doc-model's `subtree()` collected children with `stack.push(...children)`. Spreading passes every child as an *argument*, and about 130 000 of them overflow the call stack: `RangeError`, caught by the net, reported as `malformed_doc`. A loop pushes one at a time and has no such limit. The generator's own walk was already iterative; the helper it called was not.

FROM `packages/doc-model/src/index.ts`

```
function subtree(doc: Doc, id: string): string[] {
  const seen = new Set<string>();
  const stack = [id];
  for (let next = stack.pop(); next !== undefined; next = stack.pop()) {
    if (seen.has(next)) continue;
    seen.add(next);
    // A loop, not `push(...children)`: spreading passes every child as an ARGUMENT, and about
    130 000
    // of them overflow the call stack (measured by the E4.1 verifier).
    for (const child of nodeOf(doc, next)?.children ?? []) stack.push(child);
  }
  return [...seen];
}
```

FROM `packages/codegen/src/index.test.ts`

```
test("a very wide document is generated, not called malformed because a helper spread its
children into a call", () => {
  const doc = emptyDoc();
  const root = doc.nodes[ROOT_ID];
  if (!root) throw new Error("no root");
  for (let i = 0; i < 200_000; i++) {
    const id = `w${String(i)}`;
    doc.nodes[id] = { id, component: "Stack", props: {}, parentId: ROOT_ID, children: [] };
    root.children.push(id);
  }
  expect(generate(doc, manifest).ok).toBe(true);
});
```

Principal-level point: the generator trusts its inputs' *shapes* (the contract, checked by doc-model) and nothing about their *values*. Both names that become code (a component, a prop) are checked against an identifier regex after they have been found in the manifest, and every value is written from its runtime type. A schema tells you what something should be; only the value tells you what it is.

2 · A container per document, ready in a second

The sandbox is the sample app's own dev server, in a container, one per document. The image is built with the sample app as its context, so nothing of this monorepo is in it. Two facts from the learning test shape the Dockerfile: `node_modules` is *baked* at build time (first HTTP 200 in about 0.35 s, against 10 to 18 s when installed at container start), and it is installed inside Linux, because Vite's bundler is a native binary per platform.

FROM apps/worker/sandbox/Dockerfile

```
# The sandbox: the sample app's own dev server, one container per document working branch.
# Built with the SAMPLE APP as the context (`make sandbox-image`), so nothing of the monorepo is in it.
#
# node_modules is BAKED here, at build time: first HTTP 200 in about 0.35 s, against 10-18 s when
# installed at container start (learning-tests/sandbox FINDINGS 6). It is also installed INSIDE
# Linux, which matters: Vite's bundler ships a native binary per platform, so a host's
# node_modules copied in would not run.
FROM node:24-slim
RUN apt-get update && apt-get install -y --no-install-recommends git && rm -rf /var/lib/apt/lists/*
RUN npm install -g pnpm@9.15.4 && mkdir /app && chown node:node /app
USER node
WORKDIR /app
COPY --chown=node:node package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile --ignore-workspace
```

Each container makes its *own clone* of the seed, on the document's working branch `noon/<id>`, checked out over the baked `node_modules`. Only on the first start: a restarted container keeps its working tree, which is the whole point of restarting rather than recreating. "Cloned" means HEAD resolves, not ".git exists", because a clone killed halfway leaves a `.git` behind.

FROM apps/worker/sandbox/Dockerfile

```
# The container's OWN clone, on the document's working branch, checked out over the baked
# node_modules (git refuses to clone into a non-empty directory; init + fetch + checkout does not
# mind, and node_modules is in the seed's .gitignore). Only on the FIRST start: a restarted container
# keeps its working tree, which is the whole point of restarting rather than recreating it.
#
# One command per line, NOT an `a && b && c` chain: `set -e` ignores a failure anywhere in such a
# chain except its last command, so a clone that failed went on to start a dev server with no
# source (caught by the sandbox integration test). "Cloned" means HEAD resolves, not ".git exists":
# a clone killed halfway leaves a .git behind, and the next start must try again, not serve it.
# BRANCH (noon/<document id>) and PAGE are set by the worker, which has already checked the uuid
# inside them.
#
# Then the PREVIEW, on every start: an entry of our own that renders the document's page (the
# customer's own main.tsx renders their app, not our page). It lives in noon-preview/, which is
# git-excluded, so it is never part of the working branch that ship pushes. The page file itself IS
# part of it: that is the one file the document owns (keystone 8). Until the first push it holds a
# page that renders nothing, so the entry never imports a file that is not there.
```

FROM apps/worker/sandbox/Dockerfile

```
COPY --chmod=755 <<'EOF' /home/node/start.sh
#!/bin/sh
set -eu
if ! git rev-parse --quiet --verify HEAD >/dev/null 2>&1; then
    rm -rf .git
    git init --quiet --initial-branch=main
    git remote add origin "$SEED_REPO"
    git fetch --quiet origin main
    git checkout --quiet --force -B "$BRANCH" origin/main
fi
mkdir -p noon-preview
cp /home/node/preview/index.html /home/node/preview/vite.config.mjs noon-preview/
sed -e "s|@PAGE@|../$PAGE|" -e "s|@DOC@|${BRANCH#noon/}|" /home/node/preview/main.tsx > noon-
preview/main.tsx
```

```
[ -f "$PAGE" ] || cp /home/node/preview/placeholder.tsx "$PAGE"
grep -qx 'noon-preview/' .git/info/exclude || echo 'noon-preview/' >> .git/info/exclude
exec pnpm exec vite --config noon-preview/vite.config.mjs
EOF
CMD ["/home/node/start.sh"]
```

On the worker's side, `sandbox.ts` starts one and says where it answers. Docker itself is the registry: the container's name is the document's, and there is no table that can fall out of step with what is running.

FROM `apps/worker/src/sandbox.ts`

```
/**
 * The sandbox: one container per document working branch, running the sample app's own dev server
 * (apps/worker/sandbox/Dockerfile). This file STARTS one and says where it answers. Pushing the
 * generated file into it and reaping idle ones is E4.2b.
 *
 * Docker is the registry: the container's NAME is the document's, and there is no table to fall out
 * of step with what is actually running. Who may START one is a separate question, answered twice:
 * inside this process, concurrent starts of one document share a single attempt (below); across
 * processes, only the one holding the document's unfinished `sandbox` job starts it (E4.2b's unique
 * index, the same rule as one AI run per document). Name uniqueness alone was NOT enough: ten
 * racing callers removed each other's containers (the E4.2a verifier's trace, and a test now).
 */
```

The port is chosen once. Docker's own choice (`-p 127.0.0.1::5173`) is re-rolled on every `docker start` (measured: 49755, then 49767). Vite's client heals a lost connection by polling the server it was loaded from and reloading when it answers, on the *same origin* only. So the worker picks the host port, from the document id so two documents rarely want the same one, and writes it into the container's configuration, where a restart finds it.

FROM `apps/worker/src/sandbox.ts`

```
/**
 * Starts the document's sandbox, or finds the one already running, and resolves once its dev
 * server answers. Safe to call again at any time: a running sandbox is returned as it is, a stopped
 * one is started again WITH ITS WORKING TREE and ON ITS OLD PORT.
 *
 * The port never changes while the container exists, and that is a requirement, not tidiness:
 * after a restart Vite's client reloads the page by itself, but only if the new server answers on
 * the SAME origin (SPEC §2a). Docker's own port choice ( -p 127.0.0.1::5173 ) is re-rolled on every
 * `docker start` (measured: 49755, then 49767), so the port is chosen here, once, and written into
 * the container's own configuration. (A STOPPED container does not hold its port, though: if
 * something takes it meanwhile, the sandbox comes back fresh on another one. Callers must read the
 * URL from the result every time, never keep the first one.)
 */
```

FROM `apps/worker/src/sandbox.ts`

```
/** Makes sure the container exists and is running: started again if stopped, created if missing. */
async function create(run: Run, name: string, documentId: string, image: string, ports: readonly
[number, number], pool: string): Promise<void> {
  // Candidates step through the range from a start taken from the document id: spread, so two
  // documents rarely want the same port, and stepping, so a taken port is never tried twice.
  const size = ports[1] - ports[0] + 1;
  const offset = Number.parseInt(documentId.slice(0, 8), 16) % size;
  for (let attempt = 0; attempt < 5; attempt++) {
    if (await restarted(run, name)) return;
```

```

const candidate = ports[0] + ((offset + attempt) % size);
try {
  await run("run", "--detach", "--name", name,
    "--label", `${LABEL}=${pool}`, "--label", `noon.document=${documentId}`,
    "--env", `BRANCH=noon/${documentId}`, "--env", `PAGE=${pagePath(documentId)}`,
    // Loopback only: a laptop on a shared network must not serve its previews to the room.
    "--publish", `127.0.0.1:${String(candidate)}:${String(CONTAINER_PORT)}`,
    "--network", NETWORK,
    // It runs code nobody here wrote (the customer's repo) and code generated from user input.
    // ponytail: egress (and host.docker.internal) is open, and the working tree's disk is not
    // capped. Both close behind the sandbox proxy (bead noon-9gz), before E5 runs a customer
repo.
    "--cap-drop", "ALL", "--security-opt", "no-new-privileges", "--pids-limit", "256",
    "--memory", "1g", "--memory-swap", "1g", "--cpus", "1", "--tmpfs", "/tmp:size=64m",
    image);
  return;
} catch (err) {
  // `docker run` CREATES the container before it binds the port, so a port that turns out to be
  // taken leaves a container behind under the document's name. The next turn's `restarted()`
  // finds it, fails to start it for the same reason, and removes it; then the next port is
  tried.
  // A name conflict means another process created it: the next turn finds that one.
  if (!PORT_TAKEN.test(String(err)) && !String(err).includes("is already in use by container"))
  throw err;
}
}
throw new Error(`${name}: no free port after 5 tries`);
}

```

Read the last sentence of the first comment again: *a stopped container does not hold its port*. If something takes it meanwhile, `docker start` fails and the sandbox comes back fresh on another one. The port is a preference the system keeps when it can, and the URL is read after ready, from the container that answered, every time. Section 7 is where the canvas learns to live with that.

Ready is asked from inside. The worker runs in compose, where `localhost` is not the host, and the port is published on the host's loopback only. A probe run with `docker exec` inside the container works wherever the caller is. Only the probe's own "not yet" (exit code 3) is polled; any other failure is an answer.

FROM `apps/worker/src/sandbox.ts`

```

/**
 * Polls the dev server from INSIDE the container. The worker runs in compose, where `localhost` is
 * not the host, and the sandbox's port is published on the host's loopback only: asking from inside
 * works wherever the caller is. node:24-slim has no curl; Node's own fetch does the job.
 *
 * Only the probe's OWN "no" (exit code 3, chosen so nothing else produces it) means "not yet". Any
 * other failure (the container is gone, the daemon is away) is an answer, and polling it for a
minute
 * would only hide it.
 */
async function ready(run: Run, name: string, deadline: AbortSignal): Promise<void> {
  const probe = `fetch('http://127.0.0.1:${String(CONTAINER_PORT)}').then(r => process.exit(r.ok ? 0
: 3), () => process.exit(3))`;
  for (;;) {
    try {
      await run("exec", name, "node", "-e", probe);
      return;
    } catch (err) {
      if ((err as { cause?: { code?: unknown } }).cause?.code === 3) {

```

```

    await sleep(100, undefined, { signal: deadline }); // not yet; the pause throws once the
    deadline passes
    continue;
  }
  // Asked, not guessed from the error: an exec that raced the container's death fails with an
  // empty stderr (measured), so "is not running" is not always there to be read.
  const running = await run("container", "inspect", "--format", "{{.State.Running}}",
name).catch(() => undefined);
  if (running?.trim() === "false") {
    // It EXITED: its clone failed, or the dev server crashed. Say so now, with its last words,
    // which are on stderr as often as on stdout (git's "fatal: ..." is).
    const logs = await run("logs", "--tail", "20", name).catch(() => "");
    throw new Error(`${name} exited before it was ready: ${logs.trim()}`, { cause: err });
  }
  throw err;
}
}
}
}

```

FROM `apps/worker/src/sandbox.int.test.ts`

```

test("a stopped sandbox comes back on the SAME url with its working tree, because Vite's client only
self-heals on the same origin", async () => {
  const id = newDocument();
  const before = await startSandbox(id, options);
  await docker("exec", sandboxName(id), "sh", "-c", "echo kept > kept.txt");
  await docker("kill", sandboxName(id));
  const after = await startSandbox(id, options);
  expect(after.url).toBe(before.url);
  expect((await fetch(after.url)).status).toBe(200);
  expect(await docker("exec", sandboxName(id), "cat", "kept.txt")).toBe("kept");
}, 60_000);

```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

set `-e` in a start script; one
command per line

a `Dockerfile` `RUN` chain has the
same trap; Gradle's exec tasks fail
per task

`subprocess.run(check=True)` per
call, never one shell string

the host port chosen from the
document id, stepping on
collision

the same with any container client;
Testcontainers picks random ones
on purpose (tests, not previews)

the same

a probe with `docker exec node`
`-e fetch(...)`

Testcontainers' `Wait.forHttp`,
from the host

`wait_for_logs`, or a host-side
HTTP wait

an exit code chosen so nothing
else produces it

an exit code per outcome, the same
idea

the same

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A clone that failed went on to serve an empty app.

The first start script had `git init && git remote add && git fetch && git checkout` on one line under `set -e`. The fetch failed (the test points `SEED_REPO` at a path that does not exist), and the script carried on to start Vite over nothing. `set -e` exits on a failing *statement*; inside an `&&` chain, only the last command's status is the statement's, so a failure anywhere before it is ignored by design. One command per line, and now a failed clone is a container that exits with git's words in its logs, which the start reports at once instead of polling out the deadline:

FROM `apps/worker/src/sandbox.int.test.ts`

```
test("a sandbox whose dev server can never start says so at once, with its logs, instead of
waiting out the deadline", async () => {
  const id = newDocument();
  const started = Date.now();
  // Its last words are git's, on STDERR: "fatal: '/nowhere.git' does not appear to be a git
repository".
  await expect(startSandbox(id, { ...options, image: BROKEN, readyTimeoutMs: 30_000
})).rejects.toThrow(/exited before it was ready: .*nowhere\.git\/su);
  expect(Date.now() - started).toBeLessThan(10_000);
}, 60_000);
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

An exec that raced the container's death had an EMPTY stderr.

The first ready loop decided "the container is gone" by matching Docker's error text. Under load, an exec that hit the container in the moment of its death failed with no message at all, and the loop polled it for a minute. *Ask the system, do not parse its prose*: on any failure that is not the probe's own "not yet", the loop asks `container inspect` whether it is running, and an exited container is reported with its logs, stderr included, because git's "fatal" lands there.

FROM `apps/worker/src/sandbox.int.test.ts`

```
test("a sandbox removed while it is starting fails the start at once, by name, instead of
polling out the deadline", async () => {
  const id = newDocument();
  const started = Date.now();
  const start = startSandbox(id, { ...options, image: SILENT, readyTimeoutMs: 30_000 });
  setTimeout(() => void docker("rm", "--force", sandboxName(id)).catch(() => undefined), 1500);
  await expect(start).rejects.toThrow(/No such container|is not running|exited before it was
ready/u);
  expect(Date.now() - started).toBeLessThan(8_000);
}, 60_000);
```

3 • Uniqueness is not ownership

Docker refuses a second container with the same name. It is tempting to read that as "one container per document, solved". The E4.2a verifier ran ten concurrent starts of one document, and the starts *removed each other's containers*. The trace: one caller wins `docker run`. The other nine see a name conflict, take the next turn, find the container and it is not running yet, try `docker start`, and get "port not available", because the winner holds it. The code for "port taken while stopped" then does what it says: removes the leftover and tries the next port. The leftover was the winner. Callers got URLs nothing listened on, and once document A's URL served document B.

The name stops two *containers*. It says nothing about who may *act* on the one that exists. That is a different question, answered at two scopes: inside a process, concurrent starts of one document share one attempt; across processes, only the one holding the document's unfinished `sandbox` job starts it.

FROM `apps/worker/src/sandbox.ts`

```
export function startSandbox(documentId: string, options: SandboxOptions): Promise<Sandbox> {
  // The id becomes a container name, a label and a git branch. execFile has no shell, but a
  // branch called '--upload-pack=...' is still an argument. Only a plain uuid gets past here.
  if (!UUID.test(documentId)) return Promise.reject(new Error(`not a document id:
${JSON.stringify(documentId)}`));
  if (!POOL.test(options.pool)) return Promise.reject(new Error(`not a sandbox pool name:
${JSON.stringify(options.pool)}`));
  // ponytail: a caller that joins an attempt in flight gets THAT attempt's deadline and signal. Fine
  // while the only caller per document is its one sandbox job (E4.2b).
  let attempt = starting.get(documentId);
  if (!attempt) {
    attempt = start(documentId, options).finally(() => starting.delete(documentId));
    starting.set(documentId, attempt);
  }
  return attempt;
}
```

FROM `packages/db/migrations/0008_sandbox_jobs.sql`

```
-- E4.2b: the `sandbox` queue. An unfinished sandbox job is the RIGHT to start and keep a document's
-- sandbox: exactly one process may start a given document's container, or two starts race and remove
-- each other's (measured in E4.2a). So the rule lives here, where no code path can walk around it.
create unique index jobs_one_unfinished_sandbox_per_document on jobs (document_id) where queue =
'sandbox' and status in ('queued', 'running');
-- What a running job has to tell the rest of the system before it ends: for a sandbox job, the
-- address its preview answers on (the canvas puts it in an iframe). Validated by the writer.
alter table jobs add column output jsonb;
```

The single-flight map is the whole in-process answer: a `Map` from document to the promise of its attempt, deleted in `finally`. Ten callers get the same promise, so nine of them never touch Docker. Across processes, the same partial unique index pattern as lesson 3's one AI run per document: the row is the right to start, and Postgres decides who holds it. With those two in place, the "taken by something else while stopped" branch becomes safe, and its comment says exactly under what condition:

FROM `apps/worker/src/sandbox.ts`

```
/** True when the container exists and is now running; false when there is none (any leftover
removed). */
```

```

async function restarted(run: Run, name: string): Promise<boolean> {
  let running: string;
  try {
    running = (await run("container", "inspect", "--format", "{{.State.Running}}", name)).trim();
  } catch (err) {
    if (String(err).includes("No such container")) return false;
    throw err;
  }
  if (running === "true") return true;
  try {
    await run("start", name);
    return true;
  } catch (err) {
    // Its port was taken while it was stopped. The working tree goes with it: a fresh clone on a new
    // port beats a sandbox that can never start. Safe only because no one else is starting this
    // document right now (see the top of the file): otherwise "taken" may mean taken by ITSELF.
    if (!PORT_TAKEN.test(String(err))) throw err;
    await run("rm", "--force", name);
    return false;
  }
}

```

And the URL is read *after* ready, from the container that answered, never remembered from before an await that somebody else may have finished differently:

FROM apps/worker/src/sandbox.ts

```

async function start(documentId: string, options: SandboxOptions): Promise<Sandbox> {
  const { image, docker = "docker", ports = [20000, 20999], readyTimeoutMs = 60_000 } = options;
  const deadline = AbortSignal.any([AbortSignal.timeout(readyTimeoutMs), ...(options.signal ?
[options.signal] : [])]);
  const run: Run = (...args) => dockerCli(docker, args, deadline);
  const name = sandboxName(documentId);
  try {
    await ensureNetwork(run);
    await create(run, name, documentId, image, ports, options.pool);
    await ready(run, name, deadline);
    // The port is read AFTER the dev server answered, from the container that answered: never a
    // number remembered from before a restart that somebody else may have finished differently.
    return { container: name, url: `http://127.0.0.1:${String(await portOf(run, name))}/` };
  } catch (err) {
    // A deadline or a cancel ends every docker call with the same anonymous AbortError; say which.
    if (options.signal?.aborted) throw new Error(`${name}: start cancelled`, { cause: err });
    if (deadline.aborted) throw new Error(`${name}: not ready within ${String(readyTimeoutMs)} ms`, {
cause: err });
    throw err;
  }
}

```

FROM apps/worker/src/sandbox.int.test.ts

```

test("one container per document: ten concurrent starts all get the same sandbox, and its URL is
really it", async () => {
  // The E4.2a verifier's race: the losers of `docker run` found the winner still starting, got
  // "port not available" from `docker start` (the winner held it) and REMOVED the winner's
  container.
  const id = newDocument();
  const results = await Promise.all(Array.from({ length: 10 }, () => startSandbox(id, options)));
  await expectOneSandbox(id, results);
  expect(await startSandbox(id, options)).toEqual(results[0]);
}

```

```

}, 90_000);

test("ten concurrent starts of a KILLED sandbox bring it back once, with its working tree", async ()
=> {
  const id = newDocument();
  await startSandbox(id, options);
  await docker("exec", sandboxName(id), "sh", "-c", "echo kept > kept.txt");
  await docker("kill", sandboxName(id));
  const results = await Promise.all(Array.from({ length: 10 }, () => startSandbox(id, options)));
  await expectOneSandbox(id, results);
  expect(await docker("exec", sandboxName(id), "cat", "kept.txt")).toBe("kept");
}, 90_000);

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
a Map<id, Promise> for single-flight, deleted in finally	ConcurrentHashMap.computeIfAbsent(id, k -> CompletableFuture...); Guava's LoadingCache does it for you	a dict[str, asyncio.Task]; or an asyncio.Lock per key
the JavaScript event loop makes get-then-set atomic with no lock	two threads can both miss the map: computeIfAbsent or a lock	the same as JS under asyncio; a real lock under threads
a partial unique index as the cross-process rule	the same SQL through Flyway; JPA cannot express it	the same; Django UniqueConstraint(condition=...)
a number read after the await, not before	the same rule: a value read before a blocking call is stale after it	the same

The rule, in one line: uniqueness is not ownership. A unique name, a unique key, a unique port can tell you that two things cannot coexist. None of them tells a loser that what it found belongs to the winner. One starter per resource, at every scope that can start one.

4 · A child process needs an end from outside

Lesson 3's theme was that anything a program awaits needs an end from outside. A child process is the sharpest case, because it lives outside the event loop: a promise that never settles is a leak, but a `docker` CLI that never exits is a leak *and* a process holding the parent alive. Every docker call here ends at the start's deadline, and the abort is handled by hand.

FROM `apps/worker/src/sandbox.ts`

```

/**
 * One docker CLI call, resolving with what it printed. Every one has an end from outside, the
 * start's deadline: a wedged Docker daemon makes the CLI wait for ever. The abort is handled HERE,
 * with SIGKILL, not by execFile's own `signal` option: measured on Node 24, that sends SIGTERM
 * whatever `killSignal` says, and a CLI that ignores SIGTERM then outlives the call and keeps this
 * process's event loop alive.
 */
function dockerCli(docker: string, args: string[], deadline: AbortSignal, input?: string):
Promise<string> {
  return new Promise((resolve, reject) => {
    if (deadline.aborted) {
      reject(deadline.reason as Error);
    }
    return;
  });
}

```

```

    }
    const kill = (): void => { child.kill("SIGKILL"); };
    const child = execFile(docker, args, { encoding: "utf8" }, (err, stdout, stderr) => {
        deadline.removeEventListener("abort", kill);
        if (deadline.aborted) reject(new Error(`docker ${args[0]} ?? "": abandoned at the deadline`, {
            cause: deadline.reason }));
        else if (err) reject(new Error(`docker ${args[0]} ?? "": ${stderr.trim()} || err.message`, {
            cause: err }));
        // `docker logs` replays the container's stderr on its own: there, both streams are the answer.
        else resolve(args[0] === "logs" ? stdout + stderr : stdout);
    });
    deadline.addEventListener("abort", kill, { once: true });
    // A CLI that exits (or is killed) before reading all of `input` makes the write fail with EPIPE.
    // Unheard, that is an uncaught exception and the whole worker dies; the exit code already says
    it.
    child.stdin?.on("error", () => undefined);
    child.stdin?.end(input);
  });
}

```

Three things in twenty lines. **The abort listener is removed in the callback**, or every finished call would leave a listener on a long-lived signal. **The error names the subcommand and quotes stderr**, because "exit 1" tells nobody anything, and `docker logs` replays the container's stderr on the CLI's stderr, so for that one subcommand both streams are the answer. **stdin has an error listener**, for a reason that is a case study below.

FROM `apps/worker/src/sandbox.ts`

```

/**
 * Puts the generated file into the running sandbox: `docker exec`, measured the fastest and the most
 * reliable way (learning-tests/sandbox FINDINGS 2: `cat >` itself, zero missed updates once pushes
 * are paced; no bind mount). The target is the container's own PAGE, set when it was created, so
 * nothing from the caller reaches the shell.
 */
export async function pushPage(documentId: string, tsx: string, options: SandboxOptions):
Promise<void> {
  if (!UUID.test(documentId)) throw new Error(`not a document id: ${JSON.stringify(documentId)}`);
  const deadline = AbortSignal.any([AbortSignal.timeout(options.readyTimeoutMs ?? 10_000), ...
(options.signal ? [options.signal] : [])]);
  await dockerCli(options.docker ?? "docker", ["exec", "--interactive", sandboxName(documentId),
"sh", "-c", `cat > "$PAGE"`], deadline, tsx);
}

```

`pushPage` is how the generated file gets in: `docker exec --interactive <name> sh -c 'cat > "$PAGE"',` with the file on stdin. The target path is the container's own `PAGE` environment variable, set when it was created, so nothing from the caller reaches the shell. The learning test measured this the fastest and the most reliable of three ways in (about 20 ms; a bind mount missed updates two or three times in ten).

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>execFile</code> 's <code>signal</code> sends <code>SIGTERM</code> on abort, whatever <code>killSignal</code> says (Node 24, measured)	<code>Process.destroy()</code> is <code>SIGTERM</code> ; <code>destroyForcibly()</code> is <code>SIGKILL</code> , and you choose	<code>Popen.terminate()</code> vs <code>kill()</code> ; <code>run(timeout=)</code> kills
<code>child.kill("SIGKILL")</code> from our own abort listener	<code>onExit().orTimeout(...)</code> then <code>destroyForcibly()</code>	<code>wait_for(proc.wait(), t)</code> then <code>proc.kill()</code>
a write to a dead child's stdin emits <code>error</code> (<code>EPIPE</code>); unheard, it is an uncaught exception	an <code>IOException</code> on the <code>OutputStream</code> , where you write	<code>BrokenPipeError</code> , where you write
a live child keeps the event loop alive	a child does not keep the JVM alive; a non-daemon thread does	the interpreter exits; the child is orphaned

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A docker CLI that ignores SIGTERM outlived the call and kept the worker alive.

The first version used `execFile` 's own `signal` option with `killSignal: "SIGKILL"` . Measured on Node 24: the abort sends `SIGTERM` regardless. A CLI that ignores `SIGTERM` (a wedged daemon's client will) then outlives the call, and the event loop with it. So the abort is listened for here and the kill is explicit. The test's stub traps `TERM` and records its pid; after the deadline, `process.kill(pid, 0)` must throw `ESRCH`. The stub gets a 3 s window rather than 0.5 s, because macOS scans a brand-new executable on its first run and a stub killed before it ever ran would prove nothing.

FROM `apps/worker/src/sandbox.int.test.ts`

```
test("a docker CLI that ignores SIGTERM is killed anyway, not left running when the start gives up", async () => {
  const dir = mkdtempSync(join(tmpdir(), "stubborn-docker-"));
  const stubborn = join(dir, "docker");
  // `trap '' TERM` is inherited across exec: this sleep cannot be ended politely.
  writeFileSync(stubborn, `#!/bin/sh\nnecho $$ > ${dir}/pid\ntrap '' TERM\nexec sleep 3600\n`, {
    mode: 0o755 });
  // 3 s, not 0.5: macOS scans a brand-new executable on its first run, and a stub killed before it
  // ever ran would prove nothing (measured: no pid file at 300 ms).
  await expect(startSandbox(randomUUID(), { ...options, docker: stubborn, readyTimeoutMs: 3_000
  })).rejects.toThrow(/not ready within/u);
  const pid = Number(readFileSync(join(dir, "pid"), "utf8"));
  await new Promise((resolve) => setTimeout(resolve, 200));
  expect(() => process.kill(pid, 0)).toThrow(); // ESRCH: no such process
});
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A child that exited before reading its stdin would have killed the worker (EPIPE).

Found by the E4.2b re-verify. `pushPage` writes the file into the CLI's stdin. If the CLI exits first (the container is gone, the daemon refused, the CLI was just killed at the deadline), the write fails with EPIPE, and a stream error nobody listens for is an uncaught exception: the whole sandbox worker dies, every preview with it, over one push that the exit code had already reported as failed. One listener, and the exit code is the only reporter. The test pushes 4 MB (bigger than a pipe's buffer, so the write is still in flight when the child is gone) at a docker that exits at once, and then waits 200 ms for the exception that must not come.

FROM `apps/worker/src/sandbox.int.test.ts`

```
test("a docker that exits without reading the page fails the push, and cannot crash the process (EPIPE)", async () => {
  // Bigger than a pipe's buffer: the write is still in flight when the CLI is gone.
  const quitter = join(mkdtempSync(join(tmpdir(), "quitting-docker-")), "docker");
  writeFileSync(quitter, "#!/bin/sh\nexit 1\n", { mode: 0o755 });
  await expect(pushPage(randomUUID(), "x".repeat(4 * 1024 * 1024), { ...options, docker: quitter })).rejects.toThrow(/docker exec/u);
  await new Promise((r) => setTimeout(r, 200)); // an unhandled EPIPE would surface here and fail the file
});
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A wedged Docker daemon is abandoned at the deadline; a cancelled start ends at once.

A CLI that takes its arguments and waits for ever (the stub is `exec sleep 3600`) is ended by the deadline alone, and the error says which deadline. A cancel is a different name, because a stopping worker and a broken sandbox are different stories for the row.

FROM `apps/worker/src/sandbox.int.test.ts`

```
test("a docker that never answers is abandoned at the deadline, and a cancelled start ends at once", async () => {
  // A wedged daemon: the CLI takes the arguments and waits for ever. Only the deadline ends it.
  const hung = join(mkdtempSync(join(tmpdir(), "hung-docker-")), "docker");
  writeFileSync(hung, "#!/bin/sh\nexec sleep 3600\n", { mode: 0o755 });
  const started = Date.now();
  await expect(startSandbox(randomUUID(), { ...options, docker: hung, readyTimeoutMs: 500 })).rejects.toThrow(/not ready within 500 ms/u);
  expect(Date.now() - started).toBeLessThan(3_000);
  await expect(startSandbox(randomUUID(), { ...options, signal: AbortSignal.abort() })).rejects.toThrow(/cancelled/u);
});
```

5 • The sandbox queue: a job is the right to a container

The preview is a job, on a queue of its own, with the same machinery as an AI run: a row in `jobs` is the truth, a message is a pointer, `claim()` is one `UPDATE`, the sweep re-offers what Redis lost. What is new is what the job *is*. An AI run ends when the model does. A sandbox job runs for as long as somebody has the document open, and the row is the right to start and keep that document's container (section 3's index).

FROM `apps/worker/src/preview.ts`

```
/**
 * The 'sandbox' queue's handler: keeps ONE document's preview in step with the document for as long
 * as somebody has that document open.
 *
 * It joins the room as a peer that never writes and never sets a presence, so nothing is ever
 * stamped
 * with it and people do not see it (a welcome can still list it, under the owner's name, for up to 5
 * s). It projects from `peer.confirmed`, never from the optimistic
 * tree: that holds ops the server may still refuse, and a preview of a document that never existed
 * is worse than a preview a moment late.
 *
 * One loop, one tick. Each tick asks, in order: has the job been told to stop; has everyone left
 * (end
 * as succeeded); is the container alive (if not, start it again and report where it now answers: the
 * port can change when another document took it while this one was down); has the confirmed document
 * moved since the last push (generate, push). A loop and not events: every one of those questions
 * needs a clock anyway, and a loop has exactly one place where it ends.
 */
```

The handler joins the room as a peer that never writes and never sets a presence, and it projects from `peer.confirmed`: the optimistic tree holds ops the server may still refuse, and a preview of a document that never existed is worse than a preview a moment late. Then one loop with one tick, because every question in it needs a clock anyway and a loop has exactly one place where it ends.

FROM `apps/worker/src/preview.ts`

```
try {
  await start();
  let pushedSeq = -1;
  let pushedAt = 0;
  let checkedAt = Date.now();
  let lastSeenSomeone = Date.now();
  for (;;) {
    if (signal.aborted) {
      failIfStopping();
      return undefined;
    }
    // Closed for good (the room refused it: 4404, 4500, a protocol error): `others` would stay
    frozen
    // as it was, and a job that still "sees" someone would hold its container for ever.
    if (peer.status === "closed") throw new JobFailure("sync_unreachable");
    const now = Date.now();
    if (peer.others.length > 0) lastSeenSomeone = now;
    if (now - lastSeenSomeone >= idleMs) return undefined;
    if (now - checkedAt >= aliveEveryMs) {
      checkedAt = now;
      if (!(await isRunning(job.documentId, options))) {
        await reportUrl(job, null); // the canvas stops framing a dead port and says it is
        rebuilding
      }
    }
  }
}
```

```

        await start();
        pushedSeq = -1; // a fresh clone holds the placeholder, not the document: push whatever
the seq
    }
}
// ponytail: `paceMs` is the learning test's measured calibration, not logic: unpaced pushes
// outran Vite's watcher in the browser, while the file itself was right. Tune, don't delete.
if (peer.seq !== pushedSeq && now - pushedAt >= paceMs) {
    const seq = peer.seq;
    const generated = generate(peer.confirmed, manifest);
    // ponytail: a document that does not generate keeps the last good page on screen, and the
    // canvas is not told why. The reason is in `generated.reason` for when it should be.
    if (generated.ok) {
        try {
            await pushPage(job.documentId, generated.tsx, options);
        } catch (err) {
            // It died between the liveness check and the push (OOM, say): next tick brings it
back,
            // and the push after that brings the page. Any other failure is a real one.
            if (await isRunning(job.documentId, options)) throw err; // a stop meanwhile throws
here, and the catch below says which
            checkedAt = 0;
            continue;
        }
    }
    pushedSeq = seq;
    pushedAt = Date.now();
}
if (announce !== undefined && peer.status === "live" && pushedSeq === peer.seq) {
    await reportUrl(job, announce);
    announce = undefined;
}
await sleep(tickMs, undefined, { signal }).catch(() => undefined);
}

```

Read the order. Told to stop? Then the peer closed for good? Then anyone here? Then alive? Then moved? The push is paced 300 ms apart, which is not logic but the learning test's measured calibration: unpaced pushes outran Vite's watcher in the browser while the file itself was right. And a URL is reported *null* before a restart, so the canvas stops framing a dead port and says "rebuilding".

FROM `apps/worker/src/preview.ts`

```

/**
 * The address to announce once the sandbox HOLDS the document. Announced earlier, a frame could
load
 * the fresh clone's empty page, and a push landing while the dev server's watcher is still
scanning
 * is never seen: the frame stayed empty for good (an e2e run, under load).
 */
let announce: string | undefined;
const start = async (): Promise<void> => {
    let url: string;
    try {
        ({ url } = await startSandbox(job.documentId, options));
    } catch (err) {
        // A stop during the start lands here too; the catch around the loop turns it into what it
was.
        throw new JobFailure("sandbox_unavailable", err instanceof Error ? err.message :
String(err));
    }
}

```

```

    // A new address for every start, even on the same port: inside the canvas's opaque-origin
    frame
    // Vite cannot reconnect by itself (its client's SharedWorker is refused to origin "null"), so
    the
    // canvas must load the page afresh, and it does exactly when the address changes.
    announce = `${previewUrl(url, job.documentId)}&started=${String(Date.now())}`;
};

```

What a running job has to say before it ends goes through `report()`, validated with the contract the reader will use, and only while the job is running. And what is in use, for the reaper, is a question the database answers across every org:

FROM `packages/db/src/index.ts`

```

    async report({ jobId, orgId }, output) {
        const valid = PreviewOutput.nullable().parse(output); // the contract the reader will use,
        BEFORE the write
        if (!isId(jobId) || !isId(orgId)) return;
        await pool.query("update jobs set output = $3 where org_id = $1 and id = $2 and status =
        'running'", [orgId, jobId, JSON.stringify(valid)]);
    },
    sandboxesInUse: (graceMs) =>
        rows(
            z.object({ document_id: z.string() }).transform((r) => r.document_id),
            "select distinct document_id from jobs where queue = 'sandbox' and (status in ('queued',
            'running') or finished_at > now() - make_interval(secs => $1::float8 / 1000))",
            [graceMs],
        ),
    ),

```

The worker gained two lines for this epic and both are about a partial map. A process drains only the queues it has handlers for, and it checks the handler *before* it claims:

FROM `apps/worker/src/worker.ts`

```

/** One function per queue. It gets the job as Postgres has it, never what the queue message claims.
 */
/** `cancelled` is aborted when the user asks for the job to stop (F10): end quickly, keep what was
done. */
/** A handler may return what the job CONSUMED (F12); the worker records it against the job's org. */
/** A process drains ONLY the queues it has a handler for: the AI worker never holds the Docker
socket the sandbox needs. */
export type Handlers = Partial<Record<QueueName, (job: Job, cancelled: AbortSignal) =>
Promise<UsageAmount | undefined>>>;

```

FROM `apps/worker/src/worker.ts`

```

    async function run(data: unknown): Promise<void> {
        const ref = JobRef.parse(data);
        // A ref for a queue this process does not handle (misrouted, or forged by someone with the Redis
        // password) is left for its own worker: claiming it first would fail someone else's job as
        `internal`.
        if (handlers[ref.queue] === undefined) return;
        // Claiming is the ONLY way to start: queued -> running, in one statement. A duplicate or stale
        // message finds nothing to claim and ends here, so a job runs at most once per claim.
        const job = await jobs.claim(ref);
        if (!job) return;
    }

```

FROM apps/worker/src/worker.ts

```
/**
 * Jobs of each queue at once (default 4). Per queue, because they cost different things: an AI run
 * is a model call, a sandbox job holds a container for as long as someone has the document open.
 * One shared pool would let a burst of either starve the other.
 */
concurrency?: Partial<Record<QueueName, number>>;
```

FROM apps/worker/src/preview.int.test.ts

```
test("the preview follows the CONFIRMED document into the sandbox within 3 s, reports its URL, and
ends once everyone has left", async () => {
  const run = job();
  const human = await person(run.documentId);
  const { handle, urls } = handler();
  const ended = handle(run, never);

  await eventually(() => Promise.resolve(urls.length > 0), 30_000);
  // The document is named IN the URL: a stale iframe that reconnects to a port another document took
  // meanwhile asks that document's entry for the wrong page, and is refused (sandbox.int.test.ts).
  expect(urls[0]).toMatch(new RegExp(`^http://127\\.0\\.0\\.1:\\d+/${PREVIEW_PATH}\\`
doc=${run.documentId}&started=\\d+\\`, "u"));
  expect((await fetch(urls[0] ?? "").status).toBe(200);

  human.send(add("s", "root", "Stack"));
  human.send(add("t", "s", "Text", { value: "hello from the canvas" }));
  const took = await eventually(async () => (await pageIn(run.documentId)).includes(`value={"hello
from the canvas"}`), 3_000);
  expect(took).toBeLessThan(3_000);
  // Someone is here: the job stays, well past idleMs.
  const stillRunning = await Promise.race([ended.then(() => false), new Promise((r) => setTimeout(()
=> { r(true); }, 2_500))]);
  expect(stillRunning).toBe(true);

  human.stop();
  // Presence is forgotten after 5 s, then 1 s of nobody: the job ends by itself, as `succeeded`.
  await expect(Promise.race([ended, new Promise( (_, reject) => setTimeout(() => { reject(new
Error("still running")); }, 12_000))])).resolves.toBeUndefined();
  }, 60_000);
```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

a loop with one tick; every question has a clock

a `@Scheduled(fixedDelay)` method per question, and four clocks

an `asyncio` loop with one `sleep`, the same

project from `peer.confirmed`, never the optimistic tree

read committed, not your own uncommitted write

the same

`JobFailure("sync_unreachable")` : a name for the row

a checked exception per reason, mapped to a status

an exception class per reason

a handler map that is `Partial<Record<QueueName, ...>>`

`EnumMap` with absent keys; a `@Profile` per listener

`celery -Q sandbox` : the process names its queues

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A container that died between the liveness check and the push failed the job instead of restarting it.

The loop asks "alive?" once a second and "moved?" every tick. A container killed in between (OOM, say) made the push throw, and the throw ended the job as `internal`, with the document open and nobody left to bring the sandbox back. Now a failed push asks whether the container is running: if it is, the failure is real; if not, the liveness check is forced on the next tick, which restarts it and pushes the page after. The test's docker is a script that removes the container the first time it is asked to exec a push.

FROM `apps/worker/src/preview.int.test.ts`

```
test("a container that dies DURING a push is started again and given the page, not failed",
  async () => {
    // A docker whose first push finds the container gone: the death lands between the liveness
    check and the push.
    const dir = mkdtempSync(join(tmpdir(), "dying-docker-"));
    const dying = join(dir, "docker");
    writeFileSync(dying, `#!/bin/sh\nif [ "$1" = exec ] && [ "$2" = --interactive ] && [ ! -f
    ${dir}/died ]; then touch ${dir}/died; "${DOCKER}" rm --force "$3" >/dev/null; fi\nexec
    "${DOCKER}" "$@"\n`, { mode: 0o755 });
    const run = job();
    const human = await person(run.documentId);
    const { handle } = handler({ sandbox: { ...sandbox, docker: dying }, aliveEveryMs: 200 });
    const stop = new AbortController();
    const ended = handle(run, stop.signal);
    let failed: unknown;
    ended.catch((err: unknown) => { failed = err; });
    try {
      human.send(add("t", "root", "Text", { value: "after the death" }));
      await eventually(async () => {
        if (failed !== undefined) throw new Error("the job failed");
        return (await pageIn(run.documentId)).includes("after the death");
      }, 30_000);
    } finally {
      stop.abort();
      human.stop();
    }
    await expect(ended).resolves.toBeUndefined();
  }, 60_000);
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A peer closed for good froze `others`, and the job never idled out.

The idle rule is "nobody present for a minute", read from `peer.others`. A peer the room closes for good (4404 for a document that is not there, 4500, a protocol error) keeps its last `others`, and a job that still "sees" someone holds its container for ever. A closed peer is a named failure now, the same name lesson 3 gave the AI run when the sync server went away.

FROM `apps/worker/src/preview.int.test.ts`

```
test("a handler whose peer is closed for good fails the job by name instead of idling out or running for ever", async () => {
  const { handle } = handler({ sessions: { secret: TEST_SECRET, syncUrl: nowhere.server.url } });
  await expect(handle(job(), never)).rejects.toMatchObject({ reason: "sync_unreachable" });
}, 60_000);
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A worker claimed a job of a queue it had no handler for, and failed it as `internal`.

The handler lookup used the row's queue, after the claim. A sandbox ref delivered on the AI queue (misrouted, or forged by anyone holding the Redis password) was claimed by the AI worker, found no handler, and was finished as `failed`: someone else's job, ended by a process that could never have run it. Checking the handler before the claim leaves it queued for its own worker. The claim is still the only way to start; it is just not the first question.

FROM `apps/worker/src/run.int.test.ts`

```
// A sandbox ref delivered on the AI queue (misrouted, or forged by someone holding the Redis
// password) must not be claimed and then failed for want of a handler: it stays queued for
its own worker.
const aiQueue = new Queue("ai", { connection: connection(TEST_REDIS_URL), prefix });
await aiQueue.add("sandbox", { queue: "sandbox", jobId: job, orgId: doc.orgId });
await aiQueue.close();
await new Promise((r) => setTimeout(r, 300));
expect(await count("select count(*)::int as n from jobs where id = $1 and status = 'queued'
and started_at is null", [job])).toBe(1);
```

6 · One queue per process, one pool per stack, a reaper with an owner

Two decisions about *who runs what*, both made by a security review before the code was written. First: a worker process drains exactly one queue. The sandbox queue needs the Docker daemon, whose socket is root on the host. The AI queue runs a model's subprocess under a capability ceiling. They never share a process, because one escape from the subprocess would otherwise be a hand on the socket.

FROM apps/worker/src/config.ts

```
// ONE queue per process. The sandbox queue needs the Docker daemon, which is root on the host; the
// AI queue runs a model's subprocess. They never share a process (E4.2a security review).
WORKER_QUEUE: z.enum(["ai", "sandbox"], { error: "WORKER_QUEUE must be ai or sandbox"
}).default("ai"),
SANDBOX_IMAGE: z.string().min(1).default("noon-sandbox:dev"),
DOCKER: z.string().min(1).default("docker"),
// This stack's sandboxes: the only ones its reaper may remove (the daemon is shared with tests).
SANDBOX_POOL: z.string().regex(/^([a-z0-9]{0,39})$/u, "SANDBOX_POOL must be lowercase
letters, digits and hyphens").default("default"),
```

FROM docker-compose.yml

```
worker-sandbox:
  build: .
  command: ["node", "apps/worker/src/main.ts"]
  user: root # the socket inside the VM belongs to root
  environment:
    WORKER_QUEUE: sandbox
    DATABASE_URL: "postgres://noon_app:${APP_DB_PASSWORD:?run ./init.sh}@postgres:5432/noon"
    REDIS_URL: "redis://:${REDIS_PASSWORD:?run ./init.sh}@redis:6379"
    # It follows each document as a silent peer: its own session, signed with the sync server's
    secret.
    SESSION_TOKEN_SECRET: "${SESSION_TOKEN_SECRET:?run ./init.sh}"
    SYNC_URL: "ws://sync:3001"
    SANDBOX_IMAGE: noon-sandbox:dev # built on the host by ./init.sh (or `make sandbox-image`)
    # This stack's sandboxes, and the only ones its reaper removes: `make clean-clone` and the test
    # suites run their own sandboxes on the same Docker daemon.
    SANDBOX_POOL: "${COMPOSE_PROJECT_NAME:-noon-design-mvp}"
  volumes: ["/var/run/docker.sock:/var/run/docker.sock"]
  depends_on:
    migrate: { condition: service_completed_successfully }
    redis: { condition: service_healthy }
    sync: { condition: service_healthy }
```

FROM apps/worker/src/run.int.test.ts

```
test("a worker drains only the queues it has handlers for: the AI worker never claims a sandbox job",
  async () => {
    await worker?.close();
    worker = undefined;
    ai = record;
    const doc = await aDocument("ann@example.com");
    const job = await sandboxJob(doc);
    const aiOnly = await work(50); // handlers: ai only
    await producer.enqueue({ queue: "sandbox", jobId: job, orgId: doc.orgId });
    await new Promise((r) => setTimeout(r, 300)); // several sweeps
    expect(await count("select count(*)::int as n from jobs where id = $1 and status = 'queued' and
    started_at is null", [job])).toBe(1);
```

Second: every sandbox carries a *pool* label, and a reaper removes only its own pool's. This one was learned the hard way: the compose stack, `make clean-clone` and every test file share one Docker daemon, and the compose reaper deleted the test suite's sandboxes mid-run. Same daemon, same label, no owner.

FROM apps/worker/src/sandbox.ts

```
/**
 * Whose sandboxes these are: the label every container carries, and the only ones a reaper with
 * the same pool may remove. The compose stack, `make clean-clone` and each test file share one
 * Docker daemon; without this, one stack's reaper removed another's sandboxes (it happened).
 */
pool: string;
```

FROM apps/worker/src/main.ts

```
function sandboxHandlers(): Handlers {
  const sandbox = { image: config.sandbox.image, docker: config.sandbox.docker, pool:
config.sandbox.pool };
  // The reaper: every sandbox whose document has no sandbox job in flight, and none finished in the
  // last five minutes, is removed. Postgres says what is in use; the containers are only a cache of
  it.
  // ponytail: one timer, one sweep at a time; a lease if two sandbox workers ever run (E7).
  let reaping = false;
  reaper = setInterval(() => {
    if (reaping) return;
    reaping = true;
    reapSandboxes(async () => new Set(await db.jobStore().sandboxesInUse(5 * 60_000)), sandbox)
      .catch((err: unknown) => { log(`reap failed: ${describeError(err)}`); })
      .finally(() => { reaping = false; });
  }, 30_000);
  return {
    sandbox: createPreviewHandler({
      sessions: config.sessions, manifest, sandbox, stopping: stopping.signal, stillMember,
      reportUrl: (job, url) => db.jobStore().report({ queue: "sandbox", jobId: job.id, orgId:
job.orgId }, url === null ? null : { url }),
    }),
  };
}
```

The reaper itself is a dozen lines, and the order of its two questions is the case study below:

FROM apps/worker/src/sandbox.ts

```
/**
 * The reaper: removes every sandbox whose document is not in use, running or not, and returns those
 * documents. Only its own pool's. `inUse` asks Postgres: the truth is there, and a container is only
 a
 * cache of it. Removed, never merely stopped: a stopped container gives up its port, and a sandbox
 * restarted on a different port breaks the open iframe's self-healing.
 *
 * The containers are listed FIRST, then Postgres is asked. The other order races a start: "not in
 * use" is read, a job starts the document's sandbox, the list then includes it, and it is removed
 * mid-start. This way a listed container was there before the answer, which covers any job that made
 it.
 */
export async function reapSandboxes(inUse: () => Promise<ReadonlySet<string>>, options:
SandboxOptions): Promise<string[]> {
  checkPool(options.pool);
  const deadline = AbortSignal.any([AbortSignal.timeout(options.readyTimeoutMs ?? 30_000), ...
(options.signal ? [options.signal] : [])]);
  const docker = options.docker ?? "docker";
  const listed = await dockerCli(docker, ["ps", "--all", "--filter",
`label=${LABEL}=${options.pool}`, "--format", `${LABEL} "noon.document"`, deadline]);
  const used = await inUse();
```

```
// Only names this code made: a label that is not a uuid is left alone, never interpolated.
const idle = listed.split("\n").filter((id) => UUID.test(id) && !used.has(id));
if (idle.length > 0) await dockerCli(docker, ["rm", "--force", ...idle.map(sandboxName)],
deadline);
return idle;
}
```

FROM apps/worker/src/sandbox.int.test.ts

```
test("the reaper sweeps only its own pool: another stack's sandbox, not in use HERE, is left alone",
async () => {
  // The compose stack's reaper once removed the test suite's sandboxes: same daemon, same label.
  const theirs = newDocument();
  await startSandbox(theirs, { ...options, pool: testPool() });
  expect(await reapSandboxes(() => Promise.resolve(new Set()), options)).not.toContain(theirs);
  expect(await isRunning(theirs, options)).toBe(true);
}, 60_000);
```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

one process per queue, chosen by
WORKER_QUEUE

one deployable per role; Spring profiles
per listener

celery worker -Q ai and
-Q sandbox as two services

a noon.sandbox=<pool> label as
the owner

Testcontainers labels every container
with a session id, and ryuk reaps by it

the same, in
testcontainers-python

the reaper takes a function, so the
order of its questions is its own

a Supplier<Set<String>> for the
same reason

a callable

one reaper, one sweep at a time, on
one timer

@Scheduled with a lease (ShedLock)
once there are two instances

celery beat with a lock; E7 for
us

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

The reaper asked Postgres, then listed the containers: a job starting in between lost its container.

Found by the E4.2b re-verify. The first reaper took a `Set : main.ts` asked Postgres what was in use, then called it with the answer, and the reaper listed the containers. Between the answer and the list, a sandbox job started a document's container. The list had it; the answer did not; it was removed mid-start, and the job that owned it found a URL nothing listened on. Reverse the order and the race is gone: a container that is in the list existed before Postgres was asked, so whatever job made it had its row by then and is in the answer. Making the reaper take a *function* puts the order where it cannot be got wrong by a caller. (That bug is planted in drill 2.)

FROM `apps/worker/src/sandbox.int.test.ts`

```
test("the reaper lists the containers BEFORE it asks what is in use: a sandbox started in
between is never removed", async () => {
  // The race: Postgres answers "not in use", a job starts the document's sandbox, THEN docker
  ps lists it.
  const own = { ...options, pool: testPool() };
  const late = newDocument();
  const removed = await reapSandboxes(async () => {
    await startSandbox(late, own); // started after the list, so it is not in it
    return new Set<string>(); // ...and the in-use answer predates its job
  }, own);
  expect(removed).not.toContain(late);
  expect(await isRunning(late, own)).toBe(true);
}, 60_000);
```

Two rules from this section, each learned by losing something: every resource needs an owner label before it gets a reaper, or one environment's cleanup eats another's; and when a cleanup reads two systems that change under it, read the one it will act on first, and the one that grants exemptions second.

7 · The canvas frames the page

E4.3 is the api and the frame. The canvas `POST`s to make sure a preview is on its way, and `GET`s, once a second, where it answers. Membership only, like runs. The job row is the preview; the queue only tells a worker to look.

FROM `apps/api/src/app.ts`

```
// F15: the document's running page. The canvas POSTs to make sure a preview is on its way and
GETs,
// once a second, where it answers: the address can change when a sandbox restarts, so it is never
kept.
// MEMBERSHIP ONLY, like runs. The job row IS the preview; the queue only tells a worker to look.
app.post("/documents/:id/preview", async (c) => {
  const doc = await db.getDocumentForMember(c.req.param("id"), c.var.user.id);
  const opened = doc && (await db.forOrg(doc.orgId).openPreview({ documentId: doc.id, createdBy:
c.var.user.id }));
  if (!opened) return notFound(c);
  if (opened === "busy") return fail(c, 409, "preview_limit");
  if (opened.created) {
    // Left to the sweep if Redis is away, as a run is: the job exists, and it will be found.
    await enqueue(opened.created).catch((err: unknown) => {
```

```

    process.stderr.write(`${JSON.stringify({ level: "warn", path: c.req.path, message: `enqueue
failed, left to the sweep: ${describeError(err)} ` })}\n`);
  });
}
return c.json(opened.preview satisfies Preview, opened.created ? 201 : 200);
});
app.get("/documents/:id/preview", async (c) => {
  const doc = await db.getDocumentForMember(c.req.param("id"), c.var.user.id);
  const preview = doc && (await db.forOrg(doc.orgId).getPreview(doc.id));
  return preview ? c.json(preview satisfies Preview) : notFound(c);
});

```

Under the route, one `INSERT ... SELECT` does three things: takes the org and document from the documents table (tenancy from the row, never from the caller), counts this org's other unfinished sandbox jobs against a cap, and lets the partial unique index answer "already one" with `on conflict do nothing`. Then a *second statement* reads the preview as it now is, because a CTE would share the write's snapshot (lesson 3's cancel bug, remembered).

FROM `packages/db/src/index.ts`

```

openPreview: async ({ documentId, createdBy }) => {
  if (!orgExists || !isId(documentId) || (createdBy !== undefined && !isId(createdBy)))
return undefined;
  // `on conflict do nothing`: the document's unfinished job, if any, is the answer as it is.
  // ponytail: the per-org cap is a count inside the insert, so two opens at the same instant
  // can pass it together (a sandbox or two over, never unbounded). An advisory lock per org
  // makes it exact, if fairness ever has to be.
  const inserted = await rows(
    z.object({ id: z.string() }),
    "insert into jobs (org_id, document_id, queue, input, created_by) select d.org_id, d.id,
'sandbox', '{', $3 from documents d " +
    "where d.org_id = $1 and d.id = $2 and (select count(*) from jobs where org_id = $1 and
queue = 'sandbox' and status in ('queued', 'running') and document_id <> $2) < $4 " +
    // Not again straight after a failure: the canvas asks once a second, and a sandbox
that cannot
    // start at all would otherwise leave one row per second behind it.
    "and not exists (select 1 from jobs where org_id = $1 and document_id = $2 and queue =
'sandbox' " +
    "and status in ('failed', 'cancelled') and finished_at > now() - make_interval(secs =>
$5::float8 / 1000)) " +
    "on conflict do nothing returning id",
    [orgId, documentId, createdBy ?? null, MAX_PREVIEWS_PER_ORG, PREVIEW_RETRY_MS],
  );
  const preview = await readPreview(documentId);
  if (!preview) return undefined;
  const created = inserted[0] ? { queue: "sandbox" as const, jobId: inserted[0].id, orgId } :
undefined;
  // Nothing inserted, nothing unfinished, and not merely waiting out the retry pause: the
cap said no.
  if (!created && preview.status !== "queued" && preview.status !== "running" && !(await
coolingDown(documentId))) return "busy";
  return { preview, created };
},
getPreview: async (documentId) => (orgExists && isId(documentId) ? readPreview(documentId) :
undefined),

```

FROM packages/db/src/index.ts

```
/** A second statement, after any write: it reads the rows as they now are (a CTE would share
the write's snapshot). */
const coolingDown = async (documentId: string): Promise<boolean> =>
  (await one(
    z.object({ cooling: z.boolean() }),
    "select finished_at > now() - make_interval(secs => $3::float8 / 1000) as cooling from jobs
" +
    "where org_id = $1 and document_id = $2 and queue = 'sandbox' and status in ('failed',
'cancelled') order by finished_at desc limit 1",
    [orgId, documentId, PREVIEW_RETRY_MS],
  ))?.cooling === true;
const readPreview = async (documentId: string): Promise<Preview | undefined> => {
  const job = await one(PreviewRow, "select status, output from jobs where org_id = $1 and
document_id = $2 and queue = 'sandbox' order by created_at desc, id desc limit 1", [orgId,
documentId]);
  if (job) return job;
  return (await one(z.object({ id: z.string() }), "select id from documents where org_id = $1
and id = $2", [orgId, documentId])) ? { status: "none", url: null } : undefined;
};
```

The URL only counts while the job runs: a finished job's last address may belong to another document by now. And it is parsed on the way *out* with the same contract the canvas parses it with on the way *in*: loopback, http(s), or nothing. A row that says otherwise reads as "no URL", never as a 500 and never as a frame of another site.

FROM packages/db/src/index.ts

```
// The URL only counts while the job runs: a finished job's last address may belong to someone else
by now.
// Parsed with the canvas's own contract; a row it refuses (not loopback, say) reads as "no URL",
never a 500.
const PreviewRow = z.object({ status: z.string(), output: z.unknown() }).transform((r): Preview => {
  const url = r.status === "running" ? Preview.shape.url.safeParse(r.output as { url?: unknown } |
null)?.url : undefined;
  return Preview.parse({ status: r.status, url: url?.success ? url.data : null });
});
/**
 * After a preview FAILED (or was cancelled), this long before another may be started for that
document.
 * The canvas asks again every second while there is no preview; without this, a sandbox that cannot
start
 * (no image, no daemon) grew one jobs row per second per open canvas. A client cannot walk around
it.
 */
const PREVIEW_RETRY_MS = 10_000;
/** Sandboxes one org may hold at once (each is 1 CPU and 1 GiB while its document is open): the rest
of the pool stays for everyone else. */
const MAX_PREVIEWS_PER_ORG = 4;
```

FROM packages/contracts/src/index.ts

```
/**
 * What a running `sandbox` job reports: where the document's preview answers. Only http(s): the
 * canvas puts this in an iframe's src, and a `javascript:` URL there runs in the canvas's origin.
 */
export const PreviewOutput = z.strictObject({ url: z.url({ protocol: /^https?$/u }) });
export type PreviewOutput = z.infer<typeof PreviewOutput>;
/**
```

```

* A document's preview as the canvas reads it (F15): its newest sandbox job. `none` = never opened.
* `url` is set only while the job runs and the sandbox answers; null while it (re)starts =
"rebuilding".
* Loopback only, on the way in to the canvas too: until the per-document proxy (noon-9gz) every
* preview is http://127.0.0.1:<port>, and a row that said otherwise must never frame another site.
*/
export const Preview = z.strictObject({
  status: z.enum(["none", ...RunStatus.options]),
  // `URL.canParse` first: Zod runs a refine even when the format check before it failed, and a
  // `new URL()` that throws inside a refine makes safeParse THROW. A stored row must never be a 500.
  url: PreviewOutput.shape.url.refine((url) => URL.canParse(url) && new URL(url).hostname ===
    "127.0.0.1", "a preview answers on 127.0.0.1").nullable(),
});
export type Preview = z.infer<typeof Preview>;

```

FROM apps/api/src/preview.int.test.ts

```

test("the URL is read from the RUNNING job, as it is now; a finished job's old URL is never handed
out", async () => {
  const doc = await aDocumentIn("ann@example.com", await anOrg("ann@example.com"));
  await open("ann@example.com", doc);
  const [job] = await jobsOf(doc);
  const key = { queue: "sandbox" as const, jobId: job?.id ?? "", orgId: doc.orgId };
  await ctx.db.db.jobStore().claim(key);
  expect(await read("ann@example.com", doc)).toEqual({ status: "running", url: null }); // starting
  await ctx.db.db.jobStore().report(key, { url: `http://127.0.0.1:20001/noon-preview/?doc=${doc.id}`
});
  expect(await read("ann@example.com", doc)).toEqual({ status: "running", url:
`http://127.0.0.1:20001/noon-preview/?doc=${doc.id}` });
  await ctx.db.db.jobStore().report(key, null); // the container died: rebuilding
  expect(await read("ann@example.com", doc)).toEqual({ status: "running", url: null });
  await ctx.db.db.jobStore().report(key, { url: `http://127.0.0.1:20002/noon-preview/?doc=${doc.id}`
}); // back, on another port
  expect((await read("ann@example.com", doc)).url).toBe(`http://127.0.0.1:20002/noon-preview/?
doc=${doc.id}`);

  await ctx.db.db.jobStore().finish(key, "succeeded");
  expect(await read("ann@example.com", doc)).toEqual({ status: "succeeded", url: null });
  // Opening again after the end starts a new job.
  expect((await open("ann@example.com", doc)).status).toBe(201);
  expect((await jobsOf(doc)).map((j) => j.status)).toEqual(["succeeded", "queued"]);
});

```

The frame. Opened on request, because a preview holds a container (1 CPU, 1 GiB) for as long as the document is open and most visits only edit. The URL is asked for once a second and never kept. No URL while the job is live means the worker is restarting the container: "rebuilding". A preview that ended is simply opened again. And the iframe is *keyed* on the URL: a new address is a new page, and an iframe whose `src` changes keeps nothing worth keeping.

FROM apps/web/src/Preview.tsx

```

/**
* The running page (F15): the document's own sandbox, in an iframe.
*
* The URL is ASKED FOR once a second, never kept: a sandbox that comes back after its container died
* may answer on another port, and Vite's own self-heal only reloads the SAME origin. No URL while
the
* job is live = the worker is (re)starting the container: "rebuilding". A preview that ended (nobody

```

```

* was here for a minute, the worker restarted) is simply opened again.
*
* Opened on request, not with every canvas: a preview holds a container (1 CPU, 1 GiB) for as long
as
* the document is open, and most visits to a document only edit it.
*
* `sandbox="allow-scripts"` and NOT allow-same-origin: every preview is http://127.0.0.1:<port>, so
* without an opaque origin one document's page could read storage another left on a reused port.
* ponytail: polling. A pushed "the URL changed" (SSE) is the upgrade if a second of lag ever
matters.
*/
export function Preview({ documentId }: { documentId: string }) {
  const [shown, setShown] = useState(false);
  const [state, setState] = useState<PreviewState | "busy">();
  const hadUrl = useRef(false);

  useEffect(() => {
    if (!shown) return;
    let stopped = false;
    let timer: ReturnType<typeof setTimeout> | undefined;
    const tick = async (): Promise<void> => {
      let next: PreviewState | "busy" | undefined = await readPreview(documentId).catch(() =>
undefined); // a failed look is tried again in a second
      if (next && !live(next)) next = await openPreview(documentId).catch(() => next);
      if (stopped) return;
      if (next) setState(next);
      timer = setTimeout(() => void tick(), 1000);
    };
    void tick();
    return () => { stopped = true; clearTimeout(timer); };
  }, [documentId, shown]);

  if (!shown) return <section aria-label="Preview" className="preview"><button type="button" onClick=
{() => { setShown(true); }}>Show the running page</button></section>;

  const url = state === "busy" ? null : (state?.url ?? null);
  if (url) hadUrl.current = true;
  const words = url ? ""
    : state === "busy" ? "Your organisation already has as many previews open as it may. Close one,
and this one starts."
    : !state || live(state) ? (hadUrl.current ? "Rebuilding the preview..." : "Starting the preview...")
    : "The preview could not start. Trying again...";
  return (
    <section aria-label="Preview" className="preview">
      {/* keyed on the URL: a new address is a new page, an iframe whose src changes keeps nothing
worth keeping */}
      {url && <iframe key={url} title="Preview of this page" src={url} sandbox="allow-scripts" />}
      <p aria-live="polite" data-preview-status={state === "busy" ? "busy" : (state?.status ?? "")}>
{words}</p>
    </section>
  );
}

```

FROM apps/web/src/api.ts

```

// --- The preview (F15) -----
-
/** Makes sure the document's preview is on its way, and says how it is. "busy": the org already
holds its share of sandboxes (409). */
export async function openPreview(documentId: string): Promise<Preview | "busy"> {
  try {

```

```

    return await post(`/documents/${documentId}/preview`, Preview);
  } catch (problem) {
    if (problem instanceof Refused && problem.status === 409) return "busy";
    throw problem;
  }
}
/** Where the preview answers NOW. Parsed with the contract: only a loopback http(s) URL ever reaches
an iframe. */
export async function readPreview(documentId: string): Promise<Preview> {
  const res = await fetch(`/api/documents/${documentId}/preview`, { headers: { "x-dev-user": devUser
} });
  if (!res.ok) throw new Error(`GET preview answered ${String(res.status)}`);
  return Preview.parse(await res.json());
}

```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

a cap as a count inside the `INSERT` ; a bounded overshoot, named

`SELECT ... FOR UPDATE` on the org row makes it exact and serial

`pg_advisory_xact_lock(org)` ; the same trade

a contract applied on the way out of the store and on the way into the UI

a DTO validated twice: at the repository and at the controller

a pydantic model at both edges

polling once a second from a `useEffect` , cleaned up on unmount

the same from any client; SSE is the upgrade

the same

`<iframe key={url}>` : a new element, not a mutated one

n/a

n/a

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

Two opens in the same instant can both pass the per-org cap.

Not a bug found; a bug *declined*, and named. The count inside the `INSERT` is one statement, but two statements in the same instant each count three and each insert a fourth: an org can hold five for a moment, never unbounded. Compare the per-document rule in the same query, which is exact, because it is a unique index and Postgres decides who was first. The comment says what the ceiling is and what the upgrade is (an advisory lock per org). Drill 3's third question is about why the two rules are built differently.

FROM `apps/api/src/preview.int.test.ts`

```
test("one org holds at most 4 previews at once: the fifth document is refused by name until one ends", async () => {
  const org = await anOrg("bea@example.com");
  const docs = await Promise.all([1, 2, 3, 4, 5].map(() => aDocumentIn("bea@example.com", org)));
  for (const doc of docs.slice(0, 4)) expect((await open("bea@example.com", doc)).status).toBe(201);
  const refused = await open("bea@example.com", docs[4] as Document);
  expect(refused.status).toBe(409);
  expect(ErrorBody.parse(await refused.json())).toEqual({ error: "preview_limit" });
  // Another org is not held up by this one.
  expect((await open("cat@example.com", await aDocumentIn("cat@example.com", await anOrg("cat@example.com")))).status).toBe(201);
  // One of the four is re-opened fine (it is the same job), and once one ends, the fifth gets in.
  expect((await open("bea@example.com", docs[0] as Document)).status).toBe(200);
  const [job] = await jobsOf(docs[0] as Document);
  await ctx.db.rawQuery("update jobs set status = 'cancelled', finished_at = now() where id = $1", [job?.id]);
  expect((await open("bea@example.com", docs[4] as Document)).status).toBe(201);
});

test("a stored URL that no parser can read is 'no URL', never a 500 (a refine must not throw)", async () => {
  const doc = await aDocumentIn("ann@example.com", await anOrg("ann@example.com"));
  await open("ann@example.com", doc);
  const [job] = await jobsOf(doc);
  // Only reachable by writing the row by hand (report() refuses both), which is exactly what a migration
  // or a psql session can do. Zod runs a refine even after the format check failed.
  for (const url of ["http://127.0.0.1:99999/", "///127.0.0.1:1/"]) {
    await ctx.db.rawQuery("update jobs set status = 'running', started_at = now(), output = $2 where id = $1", [job?.id, JSON.stringify({ url })]);
    const res = await as("ann@example.com", "GET", `/documents/${doc.id}/preview`);
    expect(res.status).toBe(200);
    expect(Preview.parse(await res.json())).toEqual({ status: "running", url: null });
  }
});

test("after a failed preview the canvas cannot pile up jobs: one retry per cooldown, whoever asks", async () => {
  const doc = await aDocumentIn("ann@example.com", await anOrg("ann@example.com"));
  await open("ann@example.com", doc);
  const [first] = await jobsOf(doc);
  await ctx.db.rawQuery("update jobs set status = 'failed', finished_at = now(), error = 'sandbox_unavailable' where id = $1", [first?.id]);
  // The canvas asks again every second while there is no preview.
```

```

for (let i = 0; i < 5; i++) expect((await open("ann@example.com", doc)).status).toBe(200);
expect(await jobsOf(doc)).toHaveLength(1);
expect(await read("ann@example.com", doc)).toEqual({ status: "failed", url: null });
// Once the pause is over, it tries again.
await ctx.db.rawQuery("update jobs set finished_at = now() - interval '11 seconds' where id =
$1", [first?.id]);
expect((await open("ann@example.com", doc)).status).toBe(201);
expect(await jobsOf(doc)).toHaveLength(2);
});

```

8 • The opaque origin

This section is a browser rule that broke the design twice before it held, and neither break was visible from the code. Every preview is `http://127.0.0.1:<port>`: the same host, only the port differs. Without an opaque origin, one document's page could read storage another left on a reused port, and outlive the container that set it. So the E4.2a security review required `sandbox="allow-scripts"` with *no* `allow-same-origin`. The canvas does that. The consequences took three fixes.

First break: nothing rendered. A frame with an opaque origin sends `Origin: null`, the literal string, on every module request. Vite's default CORS allows any localhost origin and refuses everything else, "null" included. The page loaded, its first import was refused, and the frame stayed blank. Measured, in the browser. The E4.2a note said "lock `server.cors` to the canvas origin", which could never have worked: no request from inside the frame ever carries the canvas's origin. The fix is a Vite config of our own, wrapped around the customer's (which may say anything), allowing exactly "null" and no real origin. That is also *stricter* for real sites: another local dev server can no longer read this one's source.

FROM `apps/worker/sandbox/Dockerfile`

```

# The dev server's config: OURS, wrapped around the customer's (which may say anything). The port and
# host the worker depends on, and cors. The canvas frames the preview with sandbox="allow-scripts"
and
# no allow-same-origin, so the page's origin is "null" and every module request carries `Origin:
null`;
# Vite's default (any localhost origin) refuses those and the page never renders (measured). Allowing
# exactly "null", and no real origin, is also STRICTER for real sites: another local dev server can
no
# longer read this one's source. ponytail: any site's opaque-origin frame passes too, until the
# per-document hostname proxy (noon-9gz) puts an unguessable name in front of it.
# The customer's config is wrapped, not trusted: see the plugin below for what that takes.
COPY <<'EOF' /home/node/preview/vite.config.mjs
import { loadConfigFromFile } from "vite";

// The ONE origin that may read this dev server: the canvas's iframe, which has no origin of its own.
const ALLOWED = "null";
const IS_CORS_HEADER = /^access-control-/iu;

/**
 * Overriding `server.cors` is not enough: the customer's config can hand its source to any site
through
 * `server.headers`, or through a plugin middleware that sets the header itself (both measured by the
E4.3
 * verifier, which broke the first version of this file). Vite's own cors, forced above, writes the
header
 * for every request; this refuses every LATER attempt to change it, from a plugin or from

```

```

`server.headers`.
* Two more defences were measured and DELETED because nothing could make them matter on Node 24:
writing
* the header ourselves first (Vite's cors, with a fixed origin, already answers every request), and
* stripping the header from a `writeHead` headers object (a value set with `setHeader` wins over one
passed
* to `writeHead`, even when a plugin serves the file itself - measured, and a mutant survived both).
*/
const lockTheOrigin = () => ({
  name: "noon-preview-cors",
  enforce: "pre",
  configureServer(server) {
    server.middlewares.use((_req, res, next) => {
      const set = res.setHeader.bind(res);
      res.setHeader = (name, value) => (IS_CORS_HEADER.test(String(name)) ? res : set(name, value));
      res.appendHeader = res.setHeader;
      next();
    });
  },
});

export default async (env) => {
  const own = (await loadConfigFromFile(env, undefined, process.cwd()))?.config ?? {};
  return {
    ...own,
    plugins: [lockTheOrigin(), ...(own.plugins ?? [])],
    server: { ...own.server, host: true, port: 5173, strictPort: true, cors: { origin: ALLOWED } },
  };
};
EOF

```

FROM apps/worker/src/sandbox.int.test.ts

```

test("the preview renders inside the canvas's sandboxed iframe (an opaque origin), and no other site
may read the dev server", async () => {
  const id = newDocument();
  const sandbox = await startSandbox(id, options);
  await pushPage(id, page("framed"), options);
  const url = previewUrl(sandbox.url, id);
  // Vite answers module requests only to origins its cors allows. The frame's origin is "null"
  // (sandbox without allow-same-origin): allowed. A real site's origin: not echoed, so its reads
  fail.
  const allowed = async (origin: string): Promise<string | null> => (await fetch(new URL("/noon-
  preview/main.tsx", sandbox.url), { headers: { origin } })).headers.get("access-control-allow-
  origin");
  expect(await allowed("null")).toBe("null");
  expect(await allowed("http://evil.example")).not.toBe("http://evil.example");
  expect(await allowed("http://localhost:5173")).not.toBe("http://localhost:5173"); // Vite's default
  allowed any localhost

  const canvas = createHttpServer( (_, res) => { res.setHeader("content-type", "text/html");
  res.end(`<iframe title="preview" sandbox="allow-scripts" src="${url}"></iframe>`); });
  await new Promise<void>((resolve) => canvas.listen(0, "127.0.0.1", resolve));
  const browser = await chromium.launch();
  try {
    const tab = await browser.newPage();
    const port = (canvas.address() as { port: number }).port;
    await tab.goto(`http://localhost:${String(port)}/`); // another origin than the preview's, as the
    canvas is
    await tab.frameLocator("iframe[title=preview]").getByText("framed").waitFor({ timeout: 30_000 });
  } finally {

```

```

    await browser.close();
    canvas.close();
  }
}, 90_000);

/**
 * The customer's repo owns its vite.config, and ours is wrapped around it. These are the ways a
 * config
 * could hand the dev server's source to any site (found by the E4.3 verifier, which broke the first
 * version with the first two): the opaque-origin rule must survive all of them.
 */
const HOSTILE_CONFIGS = {
  "server.headers": `export default { server: { host: true, headers: { "Access-Control-Allow-Origin":
  "*" } } };`,
  "a plugin that echoes the origin": `export default { server: { host: true }, plugins: [{ name: "x",
  configureServer(s) { s.middlewares.use((req, res, next) => { res.setHeader("Access-Control-Allow-
  Origin", req.headers.origin ?? "*"); next(); }); } ] } };`,
  "a plugin that serves the file itself with writeHead": `export default { server: { host: true },
  plugins: [{ name: "x", configureServer(s) { s.middlewares.use((req, res, next) => { if
  (!req.url.includes("main.tsx")) return next(); res.writeHead(200, { "content-type":
  "text/javascript", "access-control-allow-origin": "*" }); res.end("export default 1"); }); } ] } };`,
  "cors wide open": `export default { server: { host: true, cors: { origin: "*" } } };`,
};

test.each(Object.entries(HOSTILE_CONFIGS))("a customer vite.config cannot open the preview to other
sites: %s", async (_name, config) => {
  const id = newDocument();
  const sandbox = await startSandbox(id, options);
  await pushPage(id, page("guarded"), options);
  const onHost = join(mkdtempSync(join(tmpdir(), "hostile-config-")), "vite.config.ts");
  writeFileSync(onHost, config);
  await docker("cp", onHost, `${sandboxName(id)}/app/vite.config.ts`);
  await docker("restart", sandboxName(id));
  const answers = async (origin: string): Promise<string | null> => {
    for (let i = 0; i < 100; i++) {
      const res = await fetch(new URL("/noon-preview/main.tsx", sandbox.url), { headers: { origin }
    }).catch(() => undefined);
      if (res?.ok) return res.headers.get("access-control-allow-origin");
      await new Promise((r) => setTimeout(r, 200));
    }
    throw new Error("the dev server never came back after the restart");
  };
  expect(await answers("http://evil.example")).toBe("null");
  expect(await answers("null")).toBe("null");
}, 120_000);

```

Second break: it never healed. The learning test showed Vite's client polling a lost server and reloading when it answered again, on the same origin, in about 1.2 s. Inside the opaque-origin frame that never happens: the client's reconnect goes through a SharedWorker, which the browser refuses to origin "null". So a restarted container on the same port would sit behind a frame that polls a server it can never rejoin. The canvas has to load a new URL, and it does exactly when the URL changes; so every start announces a distinct one.

FROM `learning-tests/sandbox/test.ts`

```

// 7. "Killing the container and starting a new one: does the browser's HMR
// client reconnect and reload on its own?"
// Previous version was VACUOUS: its loop broke on
// `title === "Hello v1" || navigated`, but the page already showed
// "Hello v1" before the kill (nothing had changed it), so it matched on

```

```
// the very first poll regardless of whether anything self-healed.
// Fixed version: pushes "Hello v2-preheal" (via docker exec, before the
// kill) so the page is showing something a fresh baked image would NOT
// show; kills+removes the container and asserts the page console
// actually logs Vite's own `[vite] server connection lost. Polling for
// restart...` message (it did, in every run); builds a NEW image
// (`lt-sandbox-v3`, built on the fly from an in-memory template, fixture
// untouched) whose Page.tsx is baked as "Hello v3-postheal" from
// container start; starts a new container from it; then polls the
// EXISTING page/tab -- without this test ever calling
// page.reload()/page.goto() -- for up to 15s.
// ACTUAL: CONFIRMED self-heals, consistently, in ~1.15-1.25s after the
// new container came up. Reading Vite's own client source
// (client.mjs, confirmed against the pinned `vite@5.4.11`): on an
// unclean WebSocket close it logs the "connection lost" message, then
// polls the server with a plain fetch every ~1s, and on the first
// successful ping calls `location.reload()` ITSELF -- so the self-heal
// is a full browser navigation, not a partial HMR patch. This test
// confirms that mechanically: `window.__marker` (set once per real page
// load in main.tsx) changed across the kill/restart in every run, i.e.
// `fullNavigationOccurred: true`. Design implication: the self-heal is
// real, but it is Vite's own client doing a full `location.reload()`
// once it can reach a server again on the same origin/port -- a canvas
// that swaps containers under a STABLE port can rely on this; one that
// changes the port or origin per container would NOT self-heal this way
// and must reload the iframe itself.
```

FROM apps/worker/src/preview.ts

```
// A new address for every start, even on the same port: inside the canvas's opaque-origin
frame
// Vite cannot reconnect by itself (its client's SharedWorker is refused to origin "null"), so
the
// canvas must load the page afresh, and it does exactly when the address changes.
announce = `${previewUrl(url, job.documentId)}&started=${String(Date.now())}`;
};
```

Third: a page left open on a reused port. A tab opened on the preview directly (not the canvas) has a real origin, and its Vite client *does* reload when its port answers again. If another document's sandbox took that port meanwhile, the reload would show that document, possibly another org's. The URL names the document it expects, and the sandbox's entry renders nothing for any other:

FROM apps/worker/src/sandbox.ts

```
/**
 * The address the canvas frames. It NAMES the document, and the sandbox's entry renders nothing for
 * another one: a page left open on a port that another document's sandbox took meanwhile (a tab
 * opened
 * on the preview directly, whose Vite client reloads when the port answers again) must not show that
 * document, possibly another org's. ponytail: until noon-9gz gives each document its own hostname.
 */
export const previewUrl = (sandboxUrl: string, documentId: string): string => new
URL(`${PREVIEW_PATH}?doc=${documentId}`, sandboxUrl).href;
```

FROM apps/worker/sandbox/Dockerfile

```
COPY <<'EOF' /home/node/preview/main.tsx
import { StrictMode } from "react";
```

```
import { createRoot } from "react-dom/client";
import "../src/design-system/tokens.css";
import { Page } from "@PAGE@";

const root = document.getElementById("root");
if (!root) throw new Error("the preview is missing #root");
// The canvas names the document it expects (?doc=). A stale iframe reconnecting to a port another
// document's sandbox took meanwhile asks for the wrong one, and gets nothing of it.
if (new URLSearchParams(location.search).get("doc") === "@DOC@") createRoot(root).render(<StrictMode>
<Page /></StrictMode>);
else root.textContent = "This preview belongs to another document. Reload the canvas.";
EOF
```

FROM `apps/worker/src/sandbox.int.test.ts`

```
// A URL naming ANOTHER document (a stale iframe, reconnected to a port this sandbox took over)
// gets no page: one document's preview never shows another's.
await tab.goto(previewUrl(sandbox.url, randomUUID()));
await tab.getByText("This preview belongs to another document").waitFor();
expect(await tab.getByText("first").count()).toBe(0);
await tab.goto(previewUrl(sandbox.url, id));
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>sandbox="allow-scripts"</code> without <code>allow-same-origin : origin "null"</code>	language-neutral; a Spring dev meets it as <code>Origin: null</code> in <code>CorsConfiguration</code>	the same in <code>django-cors-headers</code>
<code>cors: { origin: "null" }</code> , injected around the customer's config	an <code>allowedOrigins</code> list that names the literal "null"; never <code>*</code> with credentials	<code>CORS_ALLOWED_ORIGINS =</code> <code>["null"]</code>
a per-start <code>&started=</code> so the frame reloads	a cache-buster with a purpose: a new element, not a fresh fetch	the same
<code>?doc=</code> checked by the page against what it was built for	an audience check on a token: this page is for this document	the same

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

The E4.2a note said "lock cors to the canvas origin". It could never work.

It read well and it was wrong, and the way to find out was to run the frame in a real browser and watch the network tab. The lesson is not about CORS. It is that a note from a review is a hypothesis, and the fix that follows it must be *measured* against the thing it names, not implemented from the note. The ceiling of the fix is named too: any site's opaque-origin frame passes, until the per-document hostname proxy (bead noon-9gz) puts an unguessable name in front of every sandbox.

What the opaque origin bought: a preview cannot read cookies, storage or service workers, its own or anyone's, and it cannot be the canvas's origin for any purpose. What it cost: a CORS rule of our own, a reload the canvas drives, and a self-heal path (section 2's stable port) that now only matters for tabs opened on the preview directly. Both sides are written down where the next reader will look.

9 • Testing a program that runs containers

Everything in this epic touches a daemon that is root on the host, takes seconds to answer, and is shared with every other test file and the dev stack. The tests get three things right that the code alone could not.

Docker is a program on PATH, so a fake Docker is a shell script. A wedged daemon is `exec sleep 3600`; a stubborn CLI traps `TERM`; a daemon that loses the container mid-push is a script that removes it before delegating to the real CLI. No mocking library, and the code under test is the real `dockerCli`. Image variants are built from a two-line Dockerfile on stdin.

FROM `apps/worker/src/sandbox-testing.ts`

```
// Shared by the integration tests that run real sandboxes.
export const DOCKER = existsSync("/Applications/Docker.app/Contents/Resources/bin/docker") ?
"/Applications/Docker.app/Contents/Resources/bin/docker" : "docker";
// `docker build` pulls the base image through a credential helper that lives next to the CLI.
export const dockerEnv = { ...process.env, PATH: `${process.env["PATH"]} ?? "${}:${dirname(DOCKER)} ` };
export const IMAGE = "noon-sandbox:dev";
/** Each test file's own pool: a reaper, the compose stack's or another file's, never sweeps it. */
export const testPool = (): string => `test-${randomUUID().slice(0, 8)} `;
const REPO = new URL("../..../", import.meta.url).pathname;

export const docker = async (...args: string[]): Promise<string> => (await promisify(execFile)
(DOCKER, args, { timeout: 60_000, env: dockerEnv })).stdout.trim();

/** Cached after the first build, which takes minutes: it installs the sample app's dependencies. */
export async function buildImage(): Promise<void> {
  await promisify(execFile)(DOCKER, ["build", "--quiet", "--tag", IMAGE, "--file",
"apps/worker/sandbox/Dockerfile", "seed/sample-app"], { cwd: REPO, timeout: 900_000, env: dockerEnv
});
}
```

FROM `apps/worker/src/sandbox.int.test.ts`

```
beforeAll(async () => {
  await buildImage();
  // The same image, pointed at a seed repo that does not exist: its clone fails and it exits.
  await variant(BROKEN, "ENV SEED_REPO=/nowhere.git");
  // The same image, running but with no dev server: it never answers.
  await variant(SILENT, `CMD ["sleep", "infinity"]`);
}, 900_000);

async function variant(tag: string, line: string): Promise<void> {
  const build = promisify(execFile)(DOCKER, ["build", "--quiet", "--tag", tag, "-"], { timeout:
60_000, env: dockerEnv });
  build.child.stdin?.end(`FROM ${IMAGE}\n${line}\n`);
  await build;
}
```

Every file owns its pool and its port range. The pool, so no reaper (the compose one, another file's) sweeps it. The range, so a test that holds a port on the host cannot collide with another file's sandbox. The e2e layer goes one further: it runs the *real* sandbox worker from source, in a pool of its own, and stops the compose workers for the duration, because they share the jobs table and would take e2e's jobs into the compose pool.

FROM playwright.config.ts

```
// The REAL sandbox worker, from source, in a pool of its own: its reaper never touches the dev
// stack's sandboxes, nor theirs its. It prints this line once it is draining its queue.
{ command: fromEnv(`PATH="${PATH}:${DOCKER_BIN}" WORKER_QUEUE=sandbox SANDBOX_POOL=noon-e2e
DOCKER=${DOCKER_BIN}/docker SYNC_URL=ws://localhost:${String(PORTS.sync)} node
apps/worker/src/main.ts`), wait: { stdout: /worker draining queues: sandbox/u }, reuseExistingServer:
false },
```

FROM e2e/setup.ts

```
// The e2e layer shares Postgres and Redis with the dev stack, so the dev stack's WORKERS would take
// the jobs these tests create (their sweeps read the same `jobs` table): `worker` would hand runs to
// the
// real model, and `worker-sandbox` would start e2e's previews in the compose pool instead of the e2e
// worker's own. Both are stopped for the duration; teardown.ts starts them again.
// If Playwright is KILLED (kill -9), teardown never runs: `docker compose start worker worker-
// sandbox`.
// ponytail: a database and a Redis of its own for e2e is the upgrade.
/** @public Playwright loads this file by PATH (playwright.config.ts globalSetup), which the dead-
// code check cannot see. */
```

FROM e2e/preview.spec.ts

```
test("the preview follows an edit within 3 s, without reloading the page", async ({ page }) => {
  await newDocument(page);
  await button(page, "Add Button").click();
  await button(page, "Select Button 1").click();
  await labelButton(page, "First");
  await expect(preview(page).getByRole("button", { name: "First" })).toBeVisible({ timeout: 90_000
});
  // An opaque origin: scripts run, but never as 127.0.0.1 (no allow-same-origin: storage a preview
left
  // on a reused port is not another's to read).
  await expect(page.locator("iframe[title='Preview of this page']")).toHaveAttribute("sandbox",
"allow-scripts");
  // Marks THIS page load. A full reload (or a new iframe) would lose it.
  await preview(page).locator("body").evaluate(() => { (window as unknown as { noonMark: string
}).noonMark = "same load"; });

  const edited = Date.now();
  await labelButton(page, "Second");
  await expect(preview(page).getByRole("button", { name: "Second" })).toBeVisible({ timeout: 3_000
});
  expect(Date.now() - edited).toBeLessThan(3_000);
  expect(await preview(page).locator("body").evaluate(() => (window as unknown as { noonMark?: string
}).noonMark)).toBe("same load");
});
```

A self-heal test must not be vacuous. The learning test's first version passed before the kill, because the page already showed what it waited for. The e2e pushes a label a fresh container would never show, kills the container, and waits for that label to come back: only then has the new container been given the document, not merely started.

FROM e2e/preview.spec.ts

```
test.describe("when the container dies", () => {
  // The dead dev server, seen from inside the preview: its lost connection, and Vite's reconnect
```

```
code
// failing in the opaque-origin frame (a SharedWorker is refused to origin "null"; the canvas loads
the
// new address itself instead). Expected here, nowhere else.
test.use({ allowedConsole: /ERR_CONNECTION_REFUSED|ERR_EMPTY_RESPONSE|Failed to load resource|\
[vite\]|SharedWorker/u });

test("the preview says it is rebuilding, and comes back with the document on its own", async ({
page }) => {
  const documentId = await newDocument(page);
  await button(page, "Add Button").click();
  await button(page, "Select Button 1").click();
  // Something a FRESH container would not show (it starts from an empty page): if the preview
  // below shows it, the new container really was given the document, not merely started.
  await labelButton(page, "Survives");
  await expect(preview(page).getByRole("button", { name: "Survives" })).toBeVisible({ timeout:
90_000 });

  docker("rm", "--force", `noon-sandbox-${documentId}`);
  await expect(page.getByText("Rebuilding the preview...")).toBeVisible({ timeout: 10_000 });
  await expect(preview(page).getByRole("button", { name: "Survives" })).toBeVisible({ timeout:
60_000 });
  await expect(page.getByText("Rebuilding the preview...")).toBeHidden();
});
});
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
a fake docker is a shell script on PATH	Mockito on a <code>DockerClient</code> interface, or a fake binary the same way	<code>unittest.mock</code> on <code>subprocess.run</code> , or a fake binary
a pool label per test file; a port range per file	Testcontainers does the label for you; ports are random	the same
the real worker from source under Playwright, printing a line when ready	a Spring context in the test JVM; or the real jar with a health wait	the real process with <code>wait_for_logs</code>
a structural assertion (<code>node_modules</code> in the image) over a timing one	assert on the artifact, not on a stopwatch	the same

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

A flaky e2e was a bug: the URL was announced before the page was pushed.

Under load, one run of the self-heal test stayed on the fresh container's empty page for good. The handler had reported the new URL as soon as the container was ready; the canvas framed it; the frame loaded the placeholder; and the push that followed landed while Vite's watcher was still scanning the clone, so the file changed and nobody was told. Now the address is held back until the sandbox *holds* the document (the peer is live and the pushed seq is the current one), and the test records what the page file contained at the moment each URL was announced. *A flaky test is a bug report*; this was the fifth one in this project to be a real bug.

FROM apps/worker/src/preview.ts

```
if (announce !== undefined && peer.status === "live" && pushedSeq === peer.seq) {
  await reportUrl(job, announce);
  announce = undefined;
}
```

FROM apps/worker/src/preview.int.test.ts

```
await docker("rm", "--force", sandboxName(run.documentId));
// Back, with the document's page (not the placeholder a fresh clone starts with), and a URL
again.
await eventually(async () => (await pageIn(run.documentId)).includes("before the crash"),
15_000);
// In between, the URL was withdrawn: the canvas says "rebuilding" instead of framing a dead
port.
expect(urls.slice(reported)).toEqual([null,
expect.stringMatching(/^http:\/\/127\.0\.0\.1:/u)]);
// A NEW address even on the same port: Vite cannot reconnect by itself inside the canvas's
// opaque-origin frame (its client's SharedWorker is refused), so the canvas must load it
afresh.
expect(urls.at(-1)).not.toBe(urls[reported - 1]);
// Announced only once the new container HOLDS the document: a frame that loaded the fresh
clone's
// empty page first could miss the push (a watcher still scanning), and stay empty for good.
expect(pageWhenAnnounced.at(-1)).toContain("before the crash");
expect((await fetch(urls.at(-1) ?? "")).status).toBe(200);
```

CASE STUDY · FOUND BY REVIEW, BY A VERIFIER, OR BY A TEST THAT FLAKED

An 8 s bound that told "baked" from "installed" failed once under load.

Baked `node_modules` was proven by a stopwatch: about 1 s cold against 10.6 s to install, with the bound between them. At load average 117 the cold start took longer than 8 s and the test called the image unbaked. The fact being proven is structural (the directory is in the image before any container starts), so the assertion is structural now, and the stopwatch is a smoke bound only.

FROM `apps/worker/src/sandbox.int.test.ts`

```
// Baked: node_modules is IN THE IMAGE, before any container starts (a structural fact, not a
// timing guess: an 8 s bound between "about 1 s" and "10.6 s to install" failed once under
load).
await docker("run", "--rm", "--entrypoint", "test", IMAGE, "-d", "/app/node_modules/vite"); //
throws when it is not there
expect(took).toBeLessThan(30_000); // a smoke bound only
}, 60_000);
```

Never run two Docker-using suites at once. Each one's cleanup removes the other's containers and turns results into noise. It happened this session, and it is why mutation runs of these suites go in the background, one at a time.

10 · What the type system could not see, epic 4 edition

Everything below compiled under `strict`, passed the linter and the first tests.

WHAT WENT WRONG	WHY	FIX
a lying <code>toString</code> put <code>alert(1)</code> in the generated file	a value converted to a string twice answered twice	<code>typeof</code> before anything becomes code; literals from the value
<code>-0</code> was written as <code>0</code>	<code>String(-0)</code> is <code>"0"</code>	<code>Object.is(value, -0)</code>
<code>NaN</code> would have compiled as a bare identifier	<code>String(NaN)</code> is valid source	refuse non-finite numbers
a hidden prop was dropped and the file called complete	<code>Object.keys</code> and <code>Object.hasOwn</code> disagree on enumerability	compare <code>Reflect.ownKeys</code> length
a 200 000-child document was called malformed	<code>push(...children)</code> overflowed the stack at ~130k arguments	a loop
a prop set to <code>undefined</code> would have compiled without it	"present" and "has a value" differ	refuse; epic 5 would read it back as a deletion
ten concurrent starts removed each other's containers	name uniqueness is not ownership; "port taken" was the winner	single-flight in-process; a unique job across processes
a failed clone served an empty app	<code>set -e</code> ignores a failure inside <code>a && b && c</code>	one command per line
an exec racing the container's death had an empty stderr	prose was parsed	ask <code>State.Running</code>
a stubborn docker CLI outlived the deadline	Node 24's <code>signal</code> sends <code>SIGTERM</code> whatever <code>killSignal</code> says	kill with <code>SIGKILL</code> from our own listener
a docker that exited early would have killed the worker (EPIPE)	an unheard stream error is an uncaught exception	<code>child.stdin.on("error")</code>
the compose reaper deleted the test suite's sandboxes	same daemon, same label, no owner	a pool label; reap only your own
the reaper removed a sandbox that was starting	asked Postgres first, listed Docker second	list first, ask second; the reaper takes a function
a worker failed someone else's job as <code>internal</code>	the handler was looked up after the claim	check the handler before the claim
a container dying during a push failed the job	a throw is a failure, whatever caused it	ask whether it is running; force the liveness check
a closed peer held a container for ever	<code>others</code> froze at its last value	fail as <code>sync_unreachable</code>

nothing rendered in the sandboxed iframe	<code>Origin: null</code> ; Vite's default CORS refused it	an injected config allowing exactly "null"
"lock cors to the canvas origin" was a review note that could never work	the frame never sends the canvas's origin	measure the fix against the thing it names
a restarted sandbox never came back in the frame	Vite's reconnect uses a SharedWorker, refused to origin "null"	a new URL per start; the iframe keyed on it
a flaky e2e: the frame stayed on the empty page	the URL was announced before the page was pushed	announce once the sandbox holds the document
a stale tab on a reused port could show another document	the same port, a different container	<code>?doc=</code> checked by the page
an 8 s bound failed under load	a structural fact proven with a stopwatch	assert on the image, not the clock
two opens in an instant can pass the per-org cap	a count is not an index	named as a bounded overshoot; an advisory lock is the upgrade

The pattern, extended again: lessons 2 and 3 said types prove shapes and say nothing about order, time, repetition, memory, who is speaking, or what a subprocess does. Epic 4 adds three more blind spots. Types cannot see what a value's `toString` will say the second time. They cannot see who owns a resource, only that it is unique. And they cannot see a browser's security boundary: what `Origin` a request will carry, what a SharedWorker will refuse. Every one was answered the same way: measure the real thing once, then assert on what it did.

Your turn: three drills

Do these alone. They live in `drills/lesson-4/` and both coded drills start RED; `sh drills/lesson-4/check`. `sh` shows your progress.

D1 · Extend the projection, end to end. Today the generated file exists in exactly one place: inside the document's container, where the sandbox worker pushed it. Epic 5 will open a pull request with it, and an engineer will want to read it before that. Give the api `GET /documents/:id/code` : the document's generated TSX as `text/plain` , byte-for-byte what `generate()` makes from the *saved* document. You will touch the api's dependencies (it does not know codegen yet: a package that imports what it does not declare works on your machine and breaks in the image), the error contract (`not_generated` : a document the design system can no longer generate is refused by a name, never served half), and the route itself (membership only, like the preview routes; a document nobody has opened yet has no content, which is the empty document, not an error). `make check` must stay green: the gate applies to you too.

Predict first: the route reads the tree with `documentStore().load`, the same call the sync server uses. Which of section 1's promises does that rest on, and what would break if the route generated from the room's optimistic tree instead?

D2 · Fix a planted bug. `drills/lesson-4/d2-reaper/reaper.ts` is a 50-line reaper with one bug from this chapter planted in it: the order in which it asks two systems that both change under it, over a fake Docker and a fake Postgres. Its test starts a sandbox between the two questions. Form a theory from the failing assertion before you read the code; the fix moves one line.

D3 · Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. Why does the worker choose the host port itself, once, from the document id, instead of letting Docker pick a free one? And if the port never changes, why must the canvas still re-read the URL?

Because Vite's client heals a lost connection only on the *same origin*: it polls the server it was loaded from and calls `location.reload()` when it answers again (measured in the sandbox learning test). Docker's own choice (`-p 127.0.0.1:5173`) is re-rolled on every `docker start` (measured: 49755, then 49767), so a restarted container would come back on a port nobody was watching. Choosing the port here and writing it into the container's configuration makes a restart land on the same origin. It is not a guarantee, only a preference: a *stopped* container does not hold its port, and if another document (or a host process) takes it meanwhile, `docker start` fails with "port taken" and the sandbox is recreated on the next candidate. That is why the URL is read after ready, from the container that answered, and why the canvas asks once a second instead of keeping the first answer. Epic 4.3 then added a second reason: inside the canvas's opaque-origin frame Vite cannot heal at all, so every start announces a distinct URL anyway. The stable port still matters for a tab opened on the preview directly.

▼ 2. Why does a worker process drain exactly one queue (`WORKER_QUEUE=ai|sandbox`), when the code can drain both and one process would be simpler to run?

Because of what each queue needs. The sandbox queue talks to the Docker daemon, and the daemon's socket is root on the host: whoever holds it can start any container with any mount. The AI queue runs a model's subprocess, whose input is other people's documents and prompts, under a capability ceiling that took a whole section of lesson 3 to build. Put both in one process and the ceiling is decoration: one escape from the subprocess, one bug in the SDK, and the model is holding the Docker socket. So the split is not about load, it is about blast radius: `worker-sandbox` is the only compose service with `/var/run/docker.sock`, and the AI worker never has it. The handler map being *partial* is what makes this cheap: a process drains only the queues it has handlers for, and a message for the other queue is left alone before any claim. Concurrency per queue falls out too: an AI run costs a model call, a sandbox job holds 1 CPU and 1 GiB for as long as a document is open, and neither should starve the other.

▼ 3. "One unfinished sandbox job per document" is a partial unique index. "At most four previews per org" is a count inside the `INSERT`, and its comment admits a small overshoot. Why are the two rules built differently?

Because they cost different things to get exactly right, and they cost different things to get wrong. One per document is a uniqueness rule, and Postgres has a primitive for exactly that: a unique index over a predicate. One statement, no window, and Postgres decides who was first; violating it is the concurrency check. At most four per org is a *counting* rule, and there is no index for "fewer than N rows match". Making it exact means serialising every open in an org (an advisory lock on the org, or `SELECT ... FOR UPDATE` on its row), which is a queue in front of every preview for the sake of a limit that exists to protect a shared pool, not to bill anyone. A count inside the same `INSERT ... SELECT` leaves a window the width of one statement: two opens in the same instant can both count three and both pass, so an org can hold five or six for a moment, never unbounded. The rule for the per-document index is that the wrong outcome (two containers fighting over one port and removing each other) is a bug; the wrong outcome for the cap is one container too many. The comment names the ceiling and the upgrade, which is what a ponytail shortcut owes the next reader.

Recall first

Look away. Name the three things that make "one starter per document" true, each at its own scope. Then name the two systems the reaper asks and the order it must ask them in, and why. Then check.

▼ Check yourself

One starter: inside a process, `startSandbox` is single-flight (a `Map` of the attempt in flight per document, so ten callers share one); across processes, the partial unique index `jobs_one_unfinished_sandbox_per_document` means only the one process holding the document's unfinished sandbox job ever starts it; and Docker's container name is the document's, which stops two *containers* but, on its own, never stopped two starters from removing each other's. The reaper: it lists Docker first, then asks Postgres what is in use. The other way round, a job that starts a sandbox between the two answers is in the list and not in the answer, and its container is removed mid-start. Listing first means a listed container existed before the answer, so any job that made it is in the answer.

Then test it

1. A component name with a lying `toString` answered "Button" to the identifier check and code to the template. What is the fix?

Refuse anything that is not `typeof "string"` before it is looked at, and choose every literal from the value's own type

Call `toString()` once and cache the result

Escape the output with an HTML escaper

Validate the manifest with Zod at load time

A value converted twice can answer twice. The generator asks `typeof` and writes a literal from what the value IS, never from what the manifest says it is, so no `toString` is ever consulted.

2. Docker refuses a second container with the same name. Why did ten concurrent starts of one document still remove each other's containers?

A loser found the winner's container starting, got "port not available" from docker start (the winner held it), and removed it as a stale leftover

Docker's name check is eventually consistent

The image was rebuilt between the starts

The losers each got their own container on another port

Uniqueness is not ownership. "Taken" can mean taken by the winner. Fix: one starter per document, single-flight in-process and a unique job across processes.

3. A shell script starts with `set -e` and runs `git init && git fetch && git checkout` on one line. The fetch fails. What happens?

It depends on the shell

The script exits at once, as `set -e` promises

The script carries on: `set -e` ignores a failure inside an `&&` chain except in its last command

The checkout runs and then the script exits with the fetch's code

Measured: a failed clone went on to serve an empty app. One command per line, so each one is a statement `set -e` can see.

4. On Node 24 you pass `execFile(cmd, args, { signal, killSignal: "SIGKILL" })` and abort the signal. What does the child receive?

SIGKILL, as configured

SIGTERM, whatever `killSignal` says (measured); a child that ignores it outlives the call and keeps the event loop alive

SIGINT, like Ctrl-C

Nothing: the promise rejects and the child is left alone

So `dockerCli` listens for the abort itself and calls `child.kill("SIGKILL")`. The test stub traps TERM and proves the pid is gone.

5. Why is the AI worker kept from the sandbox queue by a separate process, rather than by a flag?

The AI worker has no network access to Docker

BullMQ allows one queue per connection

The sandbox queue needs the Docker socket, which is root on the host; a model's subprocess must never share a process with it

Docker CLI calls block the event loop

Blast radius, not load. `worker-sandbox` is the only compose service with the socket; the handler map is partial so a process drains only what it handles.

6. The reaper asks Postgres "which documents are in use?" and lists the containers with `docker ps`. Which order is safe, and why?

List first, then ask: a listed container existed before the answer, so any job that made it is in the answer

Ask, list, then ask again and take the intersection

Ask first, then list: the freshest container list wins

Either, as long as both happen inside one transaction

Ask-then-list lost a container to a job that started in between (found by the re-verify). `reapSandboxes` takes a function so the order is its own.

7. The canvas frames the preview with `sandbox="allow-scripts"` and no `allow-same-origin`. What did the dev server see, and what was done about it?

Every module request carried `Origin: null`; Vite's default CORS refused them and nothing rendered, so an injected config allows exactly "null" and no real origin

Vite allowed them by default because they came from localhost

Nothing changed: an iframe's requests carry the parent's origin

The requests had no Origin header, so CORS did not apply

The E4.2a note said "lock cors to the canvas origin", which could never work: requests never come from the canvas origin. Measured, then fixed; the test checks the echoed header for "null", an evil origin and localhost.

8. After a container restart, why does the worker announce a URL with a fresh `&started=` even when the port is unchanged?

To let the reaper tell old containers from new ones

To bust the browser's HTTP cache

Inside the opaque-origin frame Vite cannot reconnect by itself (its client's SharedWorker is refused to origin "null"), so the canvas must load a new URL, and it does exactly when the URL changes

Because Postgres rejects an identical output value

The iframe is keyed on the URL. A new address is a new page; the same address would have left a frame polling a server it can never rejoin.

Next: Lesson 5, after epic 5. The git peer and ship: an engineer pushes a commit that changes the generated file, and the canvas updates with ops whose actor is git; a push that breaks the fixed shape is a conflict, not a guess; and Ship turns a document into a branch and a pull request.